

SubsFlow Project Documentation

Complete Technical Blueprint, Architecture Manifest & Codebase Source Audit

Project Name:	SubsFlow (Multi-Tenant Billing Engine)
Target AI Purpose:	Full Architecture & Source Code Transfer for AI Pair Programming
Architecture Pattern:	Modular Monolith (Spring Boot 3 + PostgreSQL RLS + Kafka Outbox)
Frontend Tech:	React 18 + Vite SPA (Vanilla Glassmorphic CSS)
Backend Tech:	Java 17, Spring Boot 3.3.x, Spring Data JPA, Hibernate RLS, Flyway, Kafka
Generated Path:	c:\Users\sdona\Downloads\Springboot_Project\SubsFlow\SubsFlow_Project_Structure.pdf

1. Architecture Classification & Overview

SubsFlow is classified as a **Modular Monolith**.

Architectural Justification:

- **Single Deployment Artifact:** The entire backend runs inside a single Spring Boot application binary (SubsFlowApplication.java) listening on port 8080.
- **Logical Domain Boundaries:** The Java codebase is explicitly partitioned into 6 distinct domain packages: tenant, subscription, billing, payment, dunning, and common. Each module owns its specific JPA entities, repositories, and business services.
- **Multi-Tenancy Isolation:** Enforced natively at the database level via PostgreSQL Row-Level Security (RLS) policies using the session variable app.current_tenant_id.
- **Asynchronous Decoupling:** Domain events (such as invoice generation, payment failures) are persisted into an outbox_event table within the local transaction and published to Apache Kafka via an OutboxPublisher worker. This enables clean eventual consistency and seamless future extraction into microservices if needed.

2. Backend Module Purpose Breakdown

tenant	Manages multi-tenant onboarding, tenant status lifecycle (ACTIVE, SUSPENDED), and API key generation (sk_...). Provides public onboarding and authentication endpoints.
subscription	Core subscription engine. Manages subscriptions, plan assignments (FIXED vs USAGE_BASED), optimistic concurrency locking (@Version), status transitions (TRIAL -> ACTIVE -> PAST_DUE -> CANCELLED), and plan switches with idempotency protection.
billing	Billing cycle engine. Aggregates usage events (UsageEvent), runs automated recurring billing cycles (BillingCycleScheduler), generates detailed invoices (Invoice, InvoiceLineItem), and calculates proration credits/charges.
payment	Payment processing integration layer. Handles idempotent charge execution via IdempotencyKey (compound key: tenant_id + key), resilient payment gateway calls (PaymentGatewayResilienceService), sandbox gateway simulation, and transaction ledger (PaymentTransaction).
dunning	Automated payment retry & recovery engine. Manages failed charge retries (PaymentRetryQueue) using exponential backoff scheduling (DunningScheduler), transitioning defaulted subscriptions to PAST_DUE or CANCELLED.
common	Cross-cutting platform infrastructure: TenantContext ThreadLocal holder, TenantAuthFilter API-key security filter, Hibernate TenantConnectionProvider executing SET LOCAL app.current_tenant_id, Transactional Outbox pattern publisher/repository, Kafka config/consumers, Prometheus Micrometer metrics, and CORS configuration.

3. REST API Endpoints Registry

Method	Endpoint Path	Controller Class	Auth Required	Description
POST	/api/v1/tenants	TenantController	Public	Onboard a new tenant & generate API key
POST	/api/v1/tenants/login	TenantController	Public	Authenticate tenant via API key
GET	/api/v1/subscriptions	SubscriptionController	X-API-Key	List all tenant subscriptions
POST	/api/v1/subscriptions	SubscriptionController	X-API-Key	Create a new subscription
GET	/api/v1/subscriptions/plans	SubscriptionController	X-API-Key	List available billing plans
POST	/api/v1/subscriptions/{id}/change-plan	SubscriptionController	X-API-Key + Idempotency-Key	Switch subscription plan with proration
POST	/api/v1/subscriptions/{id}/cancel	SubscriptionController	X-API-Key	Cancel active subscription
POST	/api/v1/subscriptions/{id}/usage	SubscriptionController	X-API-Key	Ingest meter usage events
GET	/api/v1/invoices	InvoiceController	X-API-Key	List tenant invoices
GET	/api/v1/invoices/{id}	InvoiceController	X-API-Key	Get detailed invoice with line items

4. Confirmed Code & Configuration Issues Audit

1. Vite Dev Proxy IPv6 Connection Refusal (RESOLVED)

In vite.config.js, target 'http://localhost:8080' resolved to IPv6 '::1:8080' in Node 17+, causing Vite proxy to fail with 502 Bad Gateway when Tomcat listened on IPv4 127.0.0.1. Updated target to explicit 'http://127.0.0.1:8080'.

2. Public Path Matcher in TenantAuthFilter

In TenantAuthFilter.isPublicPath(), path matching explicitly checks '/api/v1/tenants' and '/api/v1/tenants/login'. Calling GET /api/v1/tenants without header returns 401 Unauthorized.

3. OutboxPublisher Polling Concurrency

OutboxPublisher uses @Scheduled(fixedRate = 5000) to poll OutboxEvent. In a multi-replica container deployment, multiple nodes could poll the same pending events simultaneously without pessimistic DB row locking (SELECT ... FOR UPDATE).

4. Dunning Retry Queue State Persistence

DunningScheduler processes payment retries. When payment retries exceed max attempts (e.g. 3 retries), the subscription status transitions to CANCELLED, but an outbox notification event should be published for webhook notification.

5. Complete Project Folder Structure

```

SubsFlow Project Workspace
■■■ Dockerfile
■■■ docker-compose.yml
■■■ pom.xml
■■■ docker/
■ ■■■ grafana/
■ ■ ■■■ dashboards/subsflow.json
■ ■ ■■■ provisioning/
■ ■ ■■■ dashboards/dashboard.yml
■ ■ ■■■ datasources/datasource.yml
■ ■■■ prometheus/prometheus.yml
■■■ frontend/
■ ■■■ index.html
■ ■■■ package.json
■ ■■■ vite.config.js
■ ■■■ src/
■ ■■■ App.css
■ ■■■ App.jsx
■ ■■■ api.js
■ ■■■ index.css
■ ■■■ main.jsx
■ ■■■ components/
■ ■■■ ApiLog.css / .jsx
■ ■■■ ChangePlanForm.jsx
■ ■■■ Dashboard.css / .jsx
■ ■■■ InvoicesPanel.css / .jsx
■ ■■■ LoginScreen.css / .jsx
■ ■■■ PlansPanel.css / .jsx
■ ■■■ SubscriptionsPanel.css / .jsx
■ ■■■ UsageForm.jsx
■■■ src/
■■■ main/
■ ■■■ java/com/subsflow/
■ ■ ■■■ SubsFlowApplication.java
■ ■ ■■■ billing/ (controller, entity, repository, scheduler, service)
■ ■ ■■■ common/ (config, context, entity, exception, filter, metrics, outbox, repository)
■ ■ ■■■ dunning/ (entity, repository, service)
■ ■ ■■■ payment/ (entity, gateway, repository, service)
■ ■ ■■■ subscription/ (controller, entity, repository, service)
■ ■ ■■■ tenant/ (controller, entity, repository)
■ ■■■ resources/
■ ■■■ application.yml
■ ■■■ db/migration/
■ ■■■ V1__init_schema.sql
■ ■■■ V2__seed_data.sql
■■■ test/java/com/subsflow/ (unit & concurrency test classes)

```

6. Source Code & Configuration Files Manifest

Below is the complete, un-summarized content of every source code and configuration file in the project. Secrets (API keys/passwords) are sanitized with [REDACTED].

File: pom.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.3.2</version>
        <relativePath/> <!-- lookup parent from repository -->
    </parent>

    <groupId>com.subsflow</groupId>
    <artifactId>subsflow</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>SubsFlow</name>
    <description>Multi-Tenant SaaS Subscription & Billing Platform</description>

    <properties>
        <java.version>17</java.version>
        <resilience4j.version>2.2.0</resilience4j.version>
    </properties>

    <dependencies>
        <!-- Spring Boot Starters -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-validation</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-redis</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-actuator</artifactId>
        </dependency>

        <!-- Kafka Event Streaming -->
        <dependency>
            <groupId>org.springframework.kafka</groupId>
            <artifactId>spring-kafka</artifactId>
        </dependency>

        <!-- Resilience4j -->
        <dependency>
            <groupId>io.github.resilience4j</groupId>
            <artifactId>resilience4j-spring-boot3</artifactId>
            <version>${resilience4j.version}</version>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-aop</artifactId>
        </dependency>

        <!-- Database & Migrations -->
        <dependency>
            <groupId>org.postgresql</groupId>
            <artifactId>postgresql</artifactId>
            <scope>runtime</scope>
        </dependency>
        <dependency>
            <groupId>org.flywaydb</groupId>
            <artifactId>flyway-core</artifactId>
        </dependency>
        <dependency>
            <groupId>org.flywaydb</groupId>
            <artifactId>flyway-database-postgresql</artifactId>
        </dependency>
    </dependencies>

```

```

<!-- Lombok -->
<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <optional>true</optional>
</dependency>

<!-- Testing Dependencies -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.springframework.kafka</groupId>
  <artifactId>spring-kafka-test</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.testcontainers</groupId>
  <artifactId>junit-jupiter</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.testcontainers</groupId>
  <artifactId>postgresql</artifactId>
  <scope>test</scope>
</dependency>
</dependencies>

<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
      <configuration>
        <excludes>
          <exclude>
            <groupId>org.projectlombok</groupId>
            <artifactId>lombok</artifactId>
          </exclude>
        </excludes>
      </configuration>
    </plugin>
  </plugins>
</build>
</project>

```

File: Dockerfile

```

FROM maven:3.9.9-eclipse-temurin-17 AS build
WORKDIR /workspace

COPY pom.xml ./
COPY src ./src

RUN mvn -B -DskipTests package

FROM eclipse-temurin:17-jre
WORKDIR /app

COPY --from=build /workspace/target/*.jar app.jar

EXPOSE 8080
ENTRYPOINT ["java", "-jar", "/app/app.jar"]

```

File: docker-compose.yml

```
version: '3.8'

services:
  app:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: subsflow-app
    depends_on:
      postgres:
        condition: service_healthy
      redis:
        condition: service_started
      kafka:
        condition: service_started
    ports:
      - "8080:8080"
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/subsflow
      SPRING_DATASOURCE_USERNAME: postgres
      SPRING_DATASOURCE_PASSWORD: [REDACTED]
      SPRING_DATA_REDIS_HOST: redis
      SPRING_KAFKA_BOOTSTRAP_SERVERS: kafka:9092

  postgres:
    image: postgres:15-alpine
    container_name: subsflow-postgres
    ports:
      - "5433:5432"
    environment:
      POSTGRES_DB: subsflow
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: [REDACTED]
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
      timeout: 5s
      retries: 5

  redis:
    image: redis:7-alpine
    container_name: subsflow-redis
    ports:
      - "6379:6379"
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 5s
      retries: 5

  zookeeper:
    image: confluentinc/cp-zookeeper:7.5.0
    container_name: subsflow-zookeeper
    ports:
      - "2181:2181"
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181
      ZOOKEEPER_TICK_TIME: 2000

  kafka:
    image: confluentinc/cp-kafka:7.5.0
    container_name: subsflow-kafka
    depends_on:
      - zookeeper
    ports:
      - "9092:9092"
    environment:
      KAFKA_BROKER_ID: 1
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
      KAFKA_LISTENERS: PLAINTEXT://0.0.0.0:9092
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
      KAFKA_AUTO_CREATE_TOPICS_ENABLE: "true"

  prometheus:
```

```
image: prom/prometheus:v2.47.0
container_name: subsflow-prometheus
ports:
  - "9090:9090"
volumes:
  - ./docker/prometheus/prometheus.yml:/etc/prometheus/prometheus.yml
command:
  - '--config.file=/etc/prometheus/prometheus.yml'

grafana:
  image: grafana/grafana:10.1.0
  container_name: subsflow-grafana
  depends_on:
    - prometheus
  ports:
    - "3000:3000"
  environment:
    GF_SECURITY_ADMIN_USER: admin
    GF_SECURITY_ADMIN_PASSWORD: [REDACTED]
  volumes:
    - ./docker/grafana/provisioning/datasources:/etc/grafana/provisioning/datasources
    - ./docker/grafana/provisioning/dashboards:/etc/grafana/provisioning/dashboards
    - ./docker/grafana/dashboards:/var/lib/grafana/dashboards

volumes:
  postgres_data:
```


File: src/main/resources/application.yml

```
spring:
  application:
    name: subsflow
  datasource:
    url: jdbc:postgresql://localhost:5433/subsflow
    username: postgres
    password: [REDACTED]
    driver-class-name: org.postgresql.Driver
  jpa:
    database-platform: org.hibernate.dialect.PostgreSQLDialect
    hibernate:
      ddl-auto: validate
    properties:
      hibernate:
        show_sql: true
        format_sql: true
  flyway:
    enabled: true
    locations: classpath:db/migration
    clean-disabled: true
  data:
    redis:
      host: localhost
      port: 6379
  kafka:
    bootstrap-servers: ${SPRING_KAFKA_BOOTSTRAP_SERVERS:localhost:9092}
    admin:
      auto-create: false
    consumer:
      group-id: subsflow-group
      auto-offset-reset: earliest
      key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
      value-deserializer: org.apache.kafka.common.serialization.StringDeserializer
    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: org.apache.kafka.common.serialization.StringSerializer
    topic:
      payment-success: payment.succeeded
      payment-failure: payment.failed
      subscription-changed: subscription.changed

management:
  endpoints:
    web:
      exposure:
        include: health, info, prometheus
  endpoint:
    health:
      show-details: always
    prometheus:
      enabled: true

server:
  port: 8080

resilience4j:
  retry:
    instances:
      paymentGateway:
        max-attempts: 2
        wait-duration: 500ms
        retry-exceptions:
          - java.io.IOException
          - java.util.concurrent.TimeoutException
  circuitbreaker:
    instances:
      paymentGateway:
        sliding-window-size: 10
        failure-rate-threshold: 50
        slow-call-rate-threshold: 50
        slow-call-duration-threshold: 2000ms
        minimum-number-of-calls: 5
        permitted-number-of-calls-in-half-open-state: 3
        wait-duration-in-open-state: 10s
        automatic-transition-from-open-to-half-open-enabled: true
```

File: src/main/resources/db/migration/V1__init_schema.sql

```
-- Enable uuid-ossf just in case we need UUID generation
CREATE EXTENSION IF NOT EXISTS "uuid-ossf";

-- 1. Tenant Table (Metadata, No RLS)
CREATE TABLE tenant (
  id VARCHAR(50) PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  status VARCHAR(50) NOT NULL,
  api_key VARCHAR(255) UNIQUE NOT NULL,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_tenant_api_key ON tenant(api_key);

-- 2. Billing Plan Table
CREATE TABLE billing_plan (
  id VARCHAR(50) PRIMARY KEY,
  tenant_id VARCHAR(50) NOT NULL,
  name VARCHAR(100) NOT NULL,
  billing_type VARCHAR(50) NOT NULL,
  price NUMERIC(20, 4) NOT NULL,
  billing_period VARCHAR(50) NOT NULL,
  version BIGINT NOT NULL DEFAULT 0,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 3. Subscription Table
CREATE TABLE subscription (
  id VARCHAR(50) PRIMARY KEY,
  tenant_id VARCHAR(50) NOT NULL,
  plan_id VARCHAR(50) NOT NULL REFERENCES billing_plan(id),
  status VARCHAR(50) NOT NULL,
  current_period_start TIMESTAMP WITH TIME ZONE NOT NULL,
  current_period_end TIMESTAMP WITH TIME ZONE NOT NULL,
  version BIGINT NOT NULL DEFAULT 0,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 4. Usage Event Table
CREATE TABLE usage_event (
  id VARCHAR(50) PRIMARY KEY,
  tenant_id VARCHAR(50) NOT NULL,
  subscription_id VARCHAR(50) NOT NULL REFERENCES subscription(id),
  quantity NUMERIC(20, 4) NOT NULL,
  event_type VARCHAR(50) NOT NULL,
  timestamp TIMESTAMP WITH TIME ZONE NOT NULL
);

-- 5. Invoice Table
CREATE TABLE invoice (
  id VARCHAR(50) PRIMARY KEY,
  tenant_id VARCHAR(50) NOT NULL,
  subscription_id VARCHAR(50) NOT NULL REFERENCES subscription(id),
  status VARCHAR(50) NOT NULL,
  amount NUMERIC(20, 4) NOT NULL,
  due_date TIMESTAMP WITH TIME ZONE NOT NULL,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 6. Invoice Line Item Table
CREATE TABLE invoice_line_item (
  id VARCHAR(50) PRIMARY KEY,
  tenant_id VARCHAR(50) NOT NULL,
  invoice_id VARCHAR(50) NOT NULL REFERENCES invoice(id) ON DELETE CASCADE,
  plan_id VARCHAR(50) REFERENCES billing_plan(id),
  type VARCHAR(50) NOT NULL,
  amount NUMERIC(20, 4) NOT NULL,
  description VARCHAR(255),
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 7. Payment Transaction Table
CREATE TABLE payment_transaction (
```

```

    id VARCHAR(50) PRIMARY KEY,
    tenant_id VARCHAR(50) NOT NULL,
    subscription_id VARCHAR(50) REFERENCES subscription(id),
    invoice_id VARCHAR(50) REFERENCES invoice(id),
    amount NUMERIC(20, 4) NOT NULL,
    status VARCHAR(50) NOT NULL,
    idempotency_key VARCHAR(255),
    gateway_reference VARCHAR(255),
    error_message TEXT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 8. Payment Retry Queue Table (Dunning)
CREATE TABLE payment_retry_queue (
    id VARCHAR(50) PRIMARY KEY,
    tenant_id VARCHAR(50) NOT NULL,
    transaction_id VARCHAR(50) NOT NULL REFERENCES payment_transaction(id),
    retry_count INT NOT NULL DEFAULT 0,
    next_retry_at TIMESTAMP WITH TIME ZONE NOT NULL,
    status VARCHAR(50) NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 9. Outbox Events Table
CREATE TABLE outbox_events (
    id VARCHAR(50) PRIMARY KEY,
    tenant_id VARCHAR(50) NOT NULL,
    event_type VARCHAR(100) NOT NULL,
    payload TEXT NOT NULL,
    status VARCHAR(50) NOT NULL DEFAULT 'PENDING',
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 10. Idempotency Key Table
CREATE TABLE idempotency_key (
    key VARCHAR(255) NOT NULL,
    tenant_id VARCHAR(50) NOT NULL,
    request_hash VARCHAR(64) NOT NULL,
    status VARCHAR(50) NOT NULL,
    response_payload TEXT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (tenant_id, key)
);

-- =====
-- Enable Row-Level Security (RLS) and Create Policies
-- =====

-- We enable RLS on all tenant-specific tables and use the custom DB session
-- variable 'app.current_tenant_id' to isolate queries.

-- billing_plan
ALTER TABLE billing_plan ENABLE ROW LEVEL SECURITY;
ALTER TABLE billing_plan FORCE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation_policy ON billing_plan FOR ALL
    USING (tenant_id = current_setting('app.current_tenant_id', true));

-- subscription
ALTER TABLE subscription ENABLE ROW LEVEL SECURITY;
ALTER TABLE subscription FORCE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation_policy ON subscription FOR ALL
    USING (tenant_id = current_setting('app.current_tenant_id', true));

-- usage_event
ALTER TABLE usage_event ENABLE ROW LEVEL SECURITY;
ALTER TABLE usage_event FORCE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation_policy ON usage_event FOR ALL
    USING (tenant_id = current_setting('app.current_tenant_id', true));

-- invoice
ALTER TABLE invoice ENABLE ROW LEVEL SECURITY;
ALTER TABLE invoice FORCE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation_policy ON invoice FOR ALL
    USING (tenant_id = current_setting('app.current_tenant_id', true));

-- invoice_line_item

```

```

ALTER TABLE invoice_line_item ENABLE ROW LEVEL SECURITY;
ALTER TABLE invoice_line_item FORCE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation_policy ON invoice_line_item FOR ALL
    USING (tenant_id = current_setting('app.current_tenant_id', true));

-- payment_transaction
ALTER TABLE payment_transaction ENABLE ROW LEVEL SECURITY;
ALTER TABLE payment_transaction FORCE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation_policy ON payment_transaction FOR ALL
    USING (tenant_id = current_setting('app.current_tenant_id', true));

-- payment_retry_queue
ALTER TABLE payment_retry_queue ENABLE ROW LEVEL SECURITY;
ALTER TABLE payment_retry_queue FORCE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation_policy ON payment_retry_queue FOR ALL
    USING (tenant_id = current_setting('app.current_tenant_id', true));

-- outbox_events
ALTER TABLE outbox_events ENABLE ROW LEVEL SECURITY;
ALTER TABLE outbox_events FORCE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation_policy ON outbox_events FOR ALL
    USING (tenant_id = current_setting('app.current_tenant_id', true));

-- idempotency_key
ALTER TABLE idempotency_key ENABLE ROW LEVEL SECURITY;
ALTER TABLE idempotency_key FORCE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation_policy ON idempotency_key FOR ALL
    USING (tenant_id = current_setting('app.current_tenant_id', true));

```

File: `src/main/resources/db/migration/V2_seed_data.sql`

```

-- =====
-- V2: Seed Data for Testing
-- =====

-- Insert test tenants
INSERT INTO tenant (id, name, status, api_key, created_at, updated_at) VALUES
    ('tenant-1', 'Acme Corp', 'ACTIVE', '[REDACTED]', CURRENT_TIMESTAMP, CURRENT_TIMESTAMP),
    ('tenant-2', 'Globex Inc', 'ACTIVE', '[REDACTED]', CURRENT_TIMESTAMP, CURRENT_TIMESTAMP);

-- Seed billing plans for tenant-1
-- The RLS policy requires app.current_tenant_id to be set for inserts.
-- Since Flyway runs as the DB owner (postgres), and we FORCE RLS,
-- we need to SET the tenant context before inserting tenant-scoped data.

SET app.current_tenant_id = 'tenant-1';

INSERT INTO billing_plan (id, tenant_id, name, billing_type, price, billing_period, version, created_at, updated_at) VALUES
    ('plan-flat-1', 'tenant-1', 'Pro Plan (Fixed)', 'FIXED', 100.0000, 'MONTHLY', 0, CURRENT_TIMESTAMP, CURRENT_TIMESTAMP),
    ('plan-usage-1', 'tenant-1', 'API Metered Plan', 'USAGE_BASED', 0.1000, 'MONTHLY', 0, CURRENT_TIMESTAMP,
CURRENT_TIMESTAMP),
    ('plan-premium-1', 'tenant-1', 'Enterprise Plan (Fixed)', 'FIXED', 500.0000, 'MONTHLY', 0, CURRENT_TIMESTAMP,
CURRENT_TIMESTAMP);

RESET app.current_tenant_id;

-- Seed billing plans for tenant-2
SET app.current_tenant_id = 'tenant-2';

INSERT INTO billing_plan (id, tenant_id, name, billing_type, price, billing_period, version, created_at, updated_at) VALUES
    ('plan-flat-2', 'tenant-2', 'Starter Plan (Fixed)', 'FIXED', 49.0000, 'MONTHLY', 0, CURRENT_TIMESTAMP, CURRENT_TIMESTAMP),
    ('plan-usage-2', 'tenant-2', 'Pay-as-you-go Plan', 'USAGE_BASED', 0.0500, 'MONTHLY', 0, CURRENT_TIMESTAMP,
CURRENT_TIMESTAMP);

RESET app.current_tenant_id;

```

File: frontend/package.json

```
{
  "name": "frontend",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "lint": "oxlint",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^19.2.8",
    "react-dom": "^19.2.8"
  },
  "devDependencies": {
    "@types/react": "^19.2.17",
    "@types/react-dom": "^19.2.3",
    "@vitejs/plugin-react": "^6.0.4",
    "oxlint": "^1.75.0",
    "vite": "^8.2.0"
  }
}
```

File: frontend/vite.config.js

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],
  server: {
    port: 5173,
    proxy: {
      '/api': {
        target: 'http://127.0.0.1:8080',
        changeOrigin: true,
        secure: false,
      },
    },
  },
})
```

File: frontendrelani.ic

```
const BASE = '/api/v1';

/**
 * Centralised API layer for SubsFlow backend.
 * Every function returns { ok, status, data } so callers get a uniform shape.
 */

async function request(path, options = {}) {
    const start = performance.now();
    const method = options.method || 'GET';
    const url = `${BASE}${path}`;

    try {
        const res = await fetch(url, {
            ...options,
            headers: {
                'Content-Type': 'application/json',
                ...(options.headers || {}),
            },
        });

        let data;
        const text = await res.text();
        try {
            data = JSON.parse(text);
        } catch {
            if (res.status === 502) {
                data = { error: '502 Bad Gateway: Backend server (localhost:8080) is unreachable. Please verify Spring Boot is running on port 8080.' };
            } else if (res.status === 503 || res.status === 504) {
                data = { error: `Server Error (${res.status}): Backend service temporarily unavailable.` };
            } else {
                data = { error: text || `HTTP ${res.status} error` };
            }
        }

        const elapsed = Math.round(performance.now() - start);

        return {
            ok: res.ok,
            status: res.status,
            data,
            meta: { method, url, elapsed },
        };
    } catch (err) {
        return {
            ok: false,
            status: 0,
            data: { error: err.message },
            meta: { method, url, elapsed: Math.round(performance.now() - start) },
        };
    }
}

/* ■■ Tenant ████████████████████████████████████████████████████████ */

export function onboardTenant(name) {
    return request('/tenants', {
        method: 'POST',
        body: JSON.stringify({ name }),
    });
}

export function loginTenant(apiKey) {
    return request('/tenants/login', {
        method: 'POST',
        body: JSON.stringify({ apiKey }),
    });
}

/* ■■ Subscriptions ████████████████████████████████████████████████████████ */

export function listSubscriptions(apiKey) {
    return request('/subscriptions', {
        headers: { 'X-API-Key': apiKey },
    });
}
```

[illegible]

File: frontendrelmain.iev

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

File: frontend/src/App.jsx

```
import { useState, useCallback } from 'react';
import LoginScreen from '../components/LoginScreen';
import Dashboard from '../components/Dashboard';
import ApiLog from '../components/ApiLog';
import './App.css';

export default function App() {
  const [tenant, setTenant] = useState(null);
  const [logs, setLogs] = useState([]);

  const addLog = useCallback((entry) => {
    setLogs((prev) => [
      {
        id: Date.now() + Math.random(),
        timestamp: new Date().toLocaleTimeString(),
        ...entry,
      },
      ...prev,
    ].slice(0, 100)); // keep last 100
  }, []);

  const handleLogout = () => {
    setTenant(null);
    setLogs([]);
  };

  return (
    <div className="app">
      { /* Ambient glow orbs */ }
      <div className="ambient-orb orb-1" />
      <div className="ambient-orb orb-2" />
      <div className="ambient-orb orb-3" />

      <header className="app-header">
        <div className="app-header-left">
          <div className="logo">
            <svg width="32" height="32" viewBox="0 0 32 32" fill="none">
              <rect width="32" height="32" rx="8" fill="url(#logo-grad)" />
              <path d="M10 16C10 12.6863 12.6863 10 16 10C19.3137 10 22 12.6863 22 16" stroke="white" strokeWidth="2.5"
strokeLinecap="round" />
              <path d="M10 20C10 16.6863 12.6863 14 16 14C19.3137 14 22 16.6863 22 20" stroke="white" strokeWidth="2.5"
strokeLinecap="round" opacity="0.5" />
              <circle cx="16" cy="22" r="2" fill="white" />
              <defs>
                <linearGradient id="logo-grad" x1="0" y1="0" x2="32" y2="32">
                  <stop stopColor="#3b82f6" />
                  <stop offset="1" stopColor="#8b5cf6" />
                </linearGradient>
              </defs>
            </svg>
          </div>
          <div>
            <h1 className="app-title">SubsFlow</h1>
            <p className="app-subtitle">Subscription Management Dashboard</p>
          </div>
        </div>
        {tenant && (
          <div className="app-header-right">
            <div className="tenant-badge">
              <span className="tenant-dot" />
              <span>{tenant.name}</span>
            </div>
            <button className="btn-logout" onClick={handleLogout}>
              Sign Out
            </button>
          </div>
        )}
      </header>

      <main className="app-main">
        {!tenant ? (
          <LoginScreen onLogin={setTenant} addLog={addLog} />
        ) : (
          <>
            <Dashboard tenant={tenant} addLog={addLog} />
            <ApiLog logs={logs} onClear={() => setLogs([])} />
          </>
        )}
      </main>
    </div>
  );
}
```



```
    }}  
  </main>  
</div>  
);  
}
```

File: frontend/src/componentell/loginScreen.jsx

```
import { useState } from 'react';
import { onboardTenant, loginTenant } from '../api';
import './LoginScreen.css';

export default function LoginScreen({ onLogin, addLog }) {
  const [mode, setMode] = useState('login'); // 'login' | 'onboard'
  const [apiKey, setApiKey] = useState('');
  const [tenantName, setTenantName] = useState('');
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState('');

  const handleLogin = async (e) => {
    e.preventDefault();
    if (!apiKey.trim()) return;
    setLoading(true);
    setError('');

    const res = await loginTenant(apiKey.trim());
    addLog({
      method: res.meta.method,
      url: res.meta.url,
      status: res.status,
      elapsed: res.meta.elapsed,
      body: res.data,
    });

    if (res.ok) {
      onLogin(res.data);
    } else {
      setError(res.data?.error || 'Login failed');
    }
    setLoading(false);
  };

  const handleOnboard = async (e) => {
    e.preventDefault();
    if (!tenantName.trim()) return;
    setLoading(true);
    setError('');

    const res = await onboardTenant(tenantName.trim());
    addLog({
      method: res.meta.method,
      url: res.meta.url,
      status: res.status,
      elapsed: res.meta.elapsed,
      body: res.data,
    });

    if (res.ok) {
      // Show the API key before logging them in
      setMode('onboarded');
      setApiKey(res.data.apiKey);
      onLogin(res.data);
    } else {
      setError(res.data?.error || 'Onboarding failed');
    }
    setLoading(false);
  };

  return (
    <div className="login-container animate-in">
      <div className="login-card glass">
        <div className="login-icon">
          <svg width="48" height="48" viewBox="0 0 48 48" fill="none">
            <rect width="48" height="48" rx="14" fill="url(#login-grad)" />
            <path d="M16 24C16 19.5817 19.5817 16 24 16C28.4183 16 32 19.5817 32 24" stroke="white" strokeWidth="3"
strokeLinecap="round" />
            <path d="M16 30C16 25.5817 19.5817 22 24 22C28.4183 22 32 25.5817 32 30" stroke="white" strokeWidth="3"
strokeLinecap="round" opacity="0.5" />
            <circle cx="24" cy="33" r="2.5" fill="white" />
            <defs>
              <linearGradient id="login-grad" x1="0" y1="0" x2="48" y2="48">
                <stop stopColor="#3b82f6" />
                <stop offset="1" stopColor="#8b5cf6" />
              </linearGradient>
            </defs>
          </svg>
        </div>
      </div>
    </div>
  );
}
```

```

    </svg>
  </div>

  <h2 className="login-title">Welcome to SubsFlow</h2>
  <p className="login-desc">Sign in with your API key or create a new tenant account.</p>

  <div className="login-tabs">
    <button
      className={`login-tab ${mode === 'login' ? 'active' : ''}`}
      onClick={() => { setMode('login'); setError(''); }}
    >
      Sign In
    </button>
    <button
      className={`login-tab ${mode === 'onboard' ? 'active' : ''}`}
      onClick={() => { setMode('onboard'); setError(''); }}
    >
      Create Tenant
    </button>
  </div>

  {mode === 'login' && (
    <form className="login-form" onSubmit={handleLogin}>
      <div className="input-group">
        <label htmlFor="api-key">API Key</label>
        <input
          id="api-key"
          className="input-field"
          type="text"
          placeholder="[REDACTED] or your API key"
          value={apiKey}
          onChange={(e) => setApiKey(e.target.value)}
          autoFocus
        />
      </div>
      {error && <p className="login-error">{error}</p>}
      <button className="btn btn-primary btn-full" type="submit" disabled={loading || !apiKey.trim()}>
        {loading ? 'Signing in...' : 'Sign In'}
      </button>
    </form>
  )}

  {mode === 'onboard' && (
    <form className="login-form" onSubmit={handleOnboard}>
      <div className="input-group">
        <label htmlFor="tenant-name">Tenant Name</label>
        <input
          id="tenant-name"
          className="input-field"
          type="text"
          placeholder="e.g. Acme Corp"
          value={tenantName}
          onChange={(e) => setTenantName(e.target.value)}
          autoFocus
        />
      </div>
      {error && <p className="login-error">{error}</p>}
      <button className="btn btn-primary btn-full" type="submit" disabled={loading || !tenantName.trim()}>
        {loading ? 'Creating...' : 'Create Tenant'}
      </button>
    </form>
  )}

  <div className="login-hint">
    <p>Test credentials: <code>[REDACTED]</code> (Acme Corp) · <code>[REDACTED]</code> (Globex Inc)</p>
  </div>
</div>
</div>
);
}

```

File: frontend/src/components/Dashboard.jsx

```
import { useState, useEffect, useCallback } from 'react';
import PlansPanel from './PlansPanel';
import SubscriptionsPanel from './SubscriptionsPanel';
import InvoicesPanel from './InvoicesPanel';
import ChangePlanForm from './ChangePlanForm';
import UsageForm from './UsageForm';
import { listPlans, listSubscriptions, listInvoices, createSubscription, cancelSubscription } from '../api';
import './Dashboard.css';

export default function Dashboard({ tenant, addLog }) {
  const [plans, setPlans] = useState([]);
  const [subscriptions, setSubscriptions] = useState([]);
  const [invoices, setInvoices] = useState([]);
  const [loadingPlans, setLoadingPlans] = useState(true);
  const [loadingSubs, setLoadingSubs] = useState(true);
  const [loadingInvoices, setLoadingInvoices] = useState(true);

  const apiKey = tenant.apiKey;

  const fetchPlans = useCallback(async () => {
    setLoadingPlans(true);
    const res = await listPlans(apiKey);
    addLog({
      method: res.meta.method,
      url: res.meta.url,
      status: res.status,
      elapsed: res.meta.elapsed,
      body: res.data,
    });
    if (res.ok) setPlans(res.data);
    setLoadingPlans(false);
  }, [apiKey, addLog]);

  const fetchSubscriptions = useCallback(async () => {
    setLoadingSubs(true);
    const res = await listSubscriptions(apiKey);
    addLog({
      method: res.meta.method,
      url: res.meta.url,
      status: res.status,
      elapsed: res.meta.elapsed,
      body: res.data,
    });
    if (res.ok) setSubscriptions(res.data);
    setLoadingSubs(false);
  }, [apiKey, addLog]);

  const fetchInvoices = useCallback(async () => {
    setLoadingInvoices(true);
    const res = await listInvoices(apiKey);
    addLog({
      method: res.meta.method,
      url: res.meta.url,
      status: res.status,
      elapsed: res.meta.elapsed,
      body: res.data,
    });
    if (res.ok) setInvoices(res.data);
    setLoadingInvoices(false);
  }, [apiKey, addLog]);

  const handleSubscribe = async (planId) => {
    const res = await createSubscription(apiKey, planId);
    addLog({
      method: res.meta.method,
      url: res.meta.url,
      status: res.status,
      elapsed: res.meta.elapsed,
      body: res.data,
    });
    if (res.ok) {
      fetchSubscriptions();
    }
  };

  const handleCancel = async (subId) => {
    const res = await cancelSubscription(apiKey, subId);
```

```

    addLog({
      method: res.meta.method,
      url: res.meta.url,
      status: res.status,
      elapsed: res.meta.elapsed,
      body: res.data,
    });
    if (res.ok) {
      fetchSubscriptions();
    }
  };

  const handlePlanChangeSuccess = () => {
    fetchSubscriptions();
    fetchInvoices();
  };

  useEffect(() => {
    fetchPlans();
    fetchSubscriptions();
    fetchInvoices();
  }, [fetchPlans, fetchSubscriptions, fetchInvoices]);

  return (
    <div className="dashboard">
      {/* Top row: Plans + Subscriptions */}
      <div className="dashboard-grid">
        <PlansPanel plans={plans} loading={loadingPlans} onRefresh={fetchPlans} onSubscribe={handleSubscribe} />
        <SubscriptionsPanel subscriptions={subscriptions} loading={loadingSubs} onRefresh={fetchSubscriptions}
onCancel={handleCancel} />
      </div>

      {/* Middle row: Invoices */}
      <InvoicesPanel
        apiKey={apiKey}
        invoices={invoices}
        loading={loadingInvoices}
        onRefresh={fetchInvoices}
        addLog={addLog}
      />

      {/* Bottom row: Actions */}
      <div className="dashboard-actions">
        <ChangePlanForm
          apiKey={apiKey}
          subscriptions={subscriptions}
          plans={plans}
          addLog={addLog}
          onSuccess={handlePlanChangeSuccess}
        />
        <UsageForm
          apiKey={apiKey}
          subscriptions={subscriptions}
          addLog={addLog}
        />
      </div>
    </div>
  );
}

```

File: `frontend/src/components/SubscriptionsPanel.jsx`

```
import './SubscriptionsPanel.css';

function statusBadgeClass(status) {
  const map = {
    ACTIVE: 'badge-active',
    TRIAL: 'badge-trial',
    PAST_DUE: 'badge-past-due',
    SUSPENDED: 'badge-suspended',
    CANCELLED: 'badge-cancelled',
  };
  return map[status] || '';
}

function formatDate(dateStr) {
  if (!dateStr) return '-';
  try {
    return new Date(dateStr).toLocaleDateString('en-US', {
      month: 'short',
      day: 'numeric',
      year: 'numeric',
    });
  } catch {
    return dateStr;
  }
}

export default function SubscriptionsPanel({ subscriptions, loading, onRefresh, onCancel }) {
  return (
    <div className="panel glass">
      <div className="section-header">
        <h2 className="section-title">
          <svg className="icon" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round">
            <path d="M12 2v4" />
            <path d="M12 18v4" />
            <path d="M4.93 4.93l2.83 2.83" />
            <path d="M16.24 16.24l2.83 2.83" />
            <path d="M2 12h4" />
            <path d="M18 12h4" />
            <path d="M4.93 19.07l2.83-2.83" />
            <path d="M16.24 7.76l2.83-2.83" />
          </svg>
          Subscriptions
        </h2>
        <button className="btn-refresh" onClick={onRefresh}>
          <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round">
            <path d="M21 2v6h-6" />
            <path d="M3 12a9 9 0 0 1 15-6.7L21 8" />
            <path d="M3 22v-6h6" />
            <path d="M21 12a9 9 0 0 1-15 6.7L3 16" />
          </svg>
          Refresh
        </button>
      </div>

      {loading ? (
        <div className="panel-loading"><div className="spinner" /></div>
      ) : subscriptions.length === 0 ? (
        <div className="panel-empty">
          <svg width="40" height="40" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="1.5">
            <circle cx="12" cy="12" r="10" />
            <path d="M8 12h8" />
          </svg>
          <p>No subscriptions found for this tenant</p>
        </div>
      ) : (
        <div className="subs-list">
          {subscriptions.map((sub) => (
            <div key={sub.id} className="sub-card">
              <div className="sub-card-top">
                <div>
                  <h3 className="sub-plan-name">{sub.planName || 'Unknown Plan'}</h3>
                  <span className="sub-id">ID: {sub.id}</span>
                </div>
                <div className="sub-card-actions">
                  <span className={ ` badge ${statusBadgeClass(sub.status)} ` }>{sub.status}</span>
                  {sub.status !== 'CANCELLED' && onCancel && (
                    <button className="btn-logout btn-cancel-sub" onClick={() => onCancel(sub.id)} title="Cancel

```

```
Subscription">
    Cancel
  </button>
  })
</div>
</div>
<div className="sub-period">
  <div className="sub-period-item">
    <span className="sub-period-label">Period Start</span>
    <span className="sub-period-value">{formatDate(sub.currentPeriodStart)}</span>
  </div>
  <div className="sub-period-divider" />
  <div className="sub-period-item">
    <span className="sub-period-label">Period End</span>
    <span className="sub-period-value">{formatDate(sub.currentPeriodEnd)}</span>
  </div>
</div>
</div>
  })
</div>
  })
</div>
);
}
```

File: frontend/src/components/PlansPanel.jsx

```
import './PlansPanel.css';

function typeBadge(billingType) {
  const map = {
    FIXED: 'badge-fixed',
    USAGE_BASED: 'badge-usage',
    TIERED: 'badge-tiered',
  };
  return map[billingType] || '';
}

function periodBadge(period) {
  return period === 'YEARLY' ? 'badge-yearly' : 'badge-monthly';
}

export default function PlansPanel({ plans, loading, onRefresh }) {
  return (
    <div className="panel glass">
      <div className="section-header">
        <h2 className="section-title">
          <svg className="icon" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round">
            <rect x="3" y="3" width="18" height="18" rx="2" />
            <path d="M3 9h18" />
            <path d="M9 21v9" />
          </svg>
          Billing Plans
        </h2>
        <button className="btn-refresh" onClick={onRefresh}>
          <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round">
            <path d="M21 2v6h-6" />
            <path d="M3 12a9 9 0 0 1 15-6.7L21 8" />
            <path d="M3 22v-6h6" />
            <path d="M21 12a9 9 0 0 1-15 6.7L3 16" />
          </svg>
          Refresh
        </button>
      </div>

      {loading ? (
        <div className="panel-loading"><div className="spinner" /></div>
      ) : plans.length === 0 ? (
        <div className="panel-empty">
          <svg width="40" height="40" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="1.5">
            <rect x="3" y="3" width="18" height="18" rx="2" />
            <path d="M3 9h18" />
          </svg>
          <p>No billing plans found</p>
        </div>
      ) : (
        <div className="plans-list">
          {plans.map((plan) => (
            <div key={plan.id} className="plan-card">
              <div className="plan-card-top">
                <h3 className="plan-name">{plan.name}</h3>
                <div className="plan-badges">
                  <span className={`badge ${typeBadge(plan.billingType)}`}>{plan.billingType.replace('_', ' ')}</span>
                  <span className={`badge ${periodBadge(plan.billingPeriod)}`}>{plan.billingPeriod}</span>
                </div>
              </div>
              <div className="plan-price">
                <span className="plan-currency">${}</span>
                <span className="plan-amount">{parseFloat(plan.price).toLocaleString()}</span>
                <span className="plan-interval">/{plan.billingPeriod === 'YEARLY' ? 'yr' : 'mo'}</span>
              </div>
              <div className="plan-card-bottom">
                <span className="plan-id">ID: {plan.id}</span>
                {onSubscribe && (
                  <button className="btn btn-sm btn-primary" onClick={() => onSubscribe(plan.id)}>
                    + Subscribe
                  </button>
                )}
              </div>
            </div>
          ))}
        </div>
      )}
    </div>
  );
}
```



```
}  
);
```

File: `frontend/src/components/InvoicesPanel.jsx`

```
import { useState } from 'react';
import { getInvoiceDetails } from '../api';
import './InvoicesPanel.css';

function invoiceStatusBadge(status) {
  const map = {
    PAID: 'badge-active',
    DRAFT: 'badge-trial',
    PENDING: 'badge-past-due',
    VOID: 'badge-cancelled',
  };
  return map[status] || '';
}

function formatDate(dateStr) {
  if (!dateStr) return '-';
  try {
    return new Date(dateStr).toLocaleDateString('en-US', {
      month: 'short',
      day: 'numeric',
      year: 'numeric',
    });
  } catch {
    return dateStr;
  }
}

export default function InvoicesPanel({ apiKey, invoices, loading, onRefresh, addLog }) {
  const [selectedInvoice, setSelectedInvoice] = useState(null);
  const [detailsLoading, setDetailsLoading] = useState(false);

  const handleSelectInvoice = async (inv) => {
    if (selectedInvoice && selectedInvoice.id === inv.id) {
      setSelectedInvoice(null);
      return;
    }

    setDetailsLoading(true);
    const res = await getInvoiceDetails(apiKey, inv.id);
    addLog({
      method: res.meta.method,
      url: res.meta.url,
      status: res.status,
      elapsed: res.meta.elapsed,
      body: res.data,
    });

    if (res.ok) {
      setSelectedInvoice(res.data);
    } else {
      setSelectedInvoice(inv);
    }
    setDetailsLoading(false);
  };

  return (
    <div className="panel glass">
      <div className="section-header">
        <h2 className="section-title">
          <svg className="icon" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round">
            <path d="M14 2H6a2 2 0 0 2 2v16a2 2 0 0 2 2h12a2 2 0 0 2 2v8z" />
            <polyline points="14 2 14 8 20 8" />
            <line x1="16" y1="13" x2="8" y2="13" />
            <line x1="16" y1="17" x2="8" y2="17" />
            <polyline points="10 9 9 9 8 9" />
          </svg>
          Invoices & Billing History
        </h2>
        <button className="btn-refresh" onClick={onRefresh}>
          <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round">
            <path d="M21 2v6h-6" />
            <path d="M3 12a9 9 0 0 1 15-6.7L21 8" />
            <path d="M3 22v-6h6" />
            <path d="M21 12a9 9 0 0 1 15 6.7L21 16" />
          </svg>
          Refresh
        </button>
      </div>
    </div>
  );
}
```

```

</div>

{loading ? (
  <div className="panel-loading"><div className="spinner" /></div>
) : invoices.length === 0 ? (
  <div className="panel-empty">
    <svg width="40" height="40" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="1.5">
      <path d="M14 2H6a2 2 0 0 0-2 2v16a2 2 0 0 0 2 2h12a2 2 0 0 0 2 2v8" />
    </svg>
    <p>No invoices generated yet</p>
  </div>
) : (
  <div className="invoices-list">
    {invoices.map((inv) => {
      const isSelected = selectedInvoice && selectedInvoice.id === inv.id;
      return (
        <div
          key={inv.id}
          className={`invoice-card ${isSelected ? 'invoice-card-selected' : ''}`}
          onClick={() => handleSelectInvoice(inv)}
        >
          <div className="invoice-card-top">
            <div>
              <h3 className="invoice-id">Invoice: {inv.id}</h3>
              <span className="invoice-sub-id">Sub: {inv.subscriptionId || 'N/A'}</span>
            </div>
            <div className="invoice-amount-box">
              <span className="invoice-amount">${parseFloat(inv.amount).toFixed(2)}</span>
              <span className={`badge ${invoiceStatusBadge(inv.status)} `}>{inv.status}</span>
            </div>
          </div>

          <div className="invoice-dates">
            <span>Created: {formatDate(inv.createdAt)}</span>
            <span>Due: {formatDate(inv.dueDate)}</span>
          </div>

          {isSelected && (
            <div className="invoice-details-expand">
              <h4 className="line-items-title">Line Items</h4>
              {detailsLoading ? (
                <div className="spinner" style={{ margin: '12px auto' }} />
              ) : selectedInvoice.lineItems && selectedInvoice.lineItems.length > 0 ? (
                <div className="line-items-list">
                  {selectedInvoice.lineItems.map((item) => (
                    <div key={item.id} className="line-item-row">
                      <div className="line-item-desc">
                        <span>{item.description}</span>
                        <span className="line-item-type">{item.type}</span>
                      </div>
                      <span className={`line-item-amount ${item.amount < 0 ? 'amount-credit' : ''}`}>
                        {item.amount < 0 ? '-' : ''}${Math.abs(parseFloat(item.amount)).toFixed(2)}
                      </span>
                    </div>
                  ))}
                </div>
              ) : (
                <p className="no-line-items">No detailed line items available.</p>
              )}
            </div>
          )}
        </div>
      )}
    )}
  </div>
);
</div>
);
}

```

File: `frontend/src/components/ChangePlanForm.jsx`

```
import { useState } from 'react';
import { changePlan } from '../api';

export default function ChangePlanForm({ apiKey, subscriptions, plans, addLog, onSuccess }) {
  const [subId, setSubId] = useState('');
  const [planId, setPlanId] = useState('');
  const [paymentMethod, setPaymentMethod] = useState('');
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState('');
  const [success, setSuccess] = useState(false);

  const handleSubmit = async (e) => {
    e.preventDefault();
    if (!subId || !planId || !paymentMethod) return;

    setLoading(true);
    setError('');
    setSuccess(false);

    // Auto-generate idempotency key
    const idempotencyKey = 'idk_' + Math.random().toString(36).substring(2, 10);

    const res = await changePlan(apiKey, subId, planId, paymentMethod, idempotencyKey);
    addLog({
      method: res.meta.method,
      url: res.meta.url,
      status: res.status,
      elapsed: res.meta.elapsed,
      body: res.data,
      idempotencyKey,
    });

    if (res.ok) {
      setSuccess(true);
      setSubId('');
      setPlanId('');
      setPaymentMethod('');
      onSuccess(); // Refresh subs
    } else {
      setError(res.data?.error || 'Failed to change plan');
    }
    setLoading(false);
  };

  return (
    <div className="panel glass">
      <div className="section-header">
        <h2 className="section-title">
          <svg className="icon" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round">
            <path d="M12 20h9" />
            <path d="M16.5 3.5a2.121 2.121 0 0 1 3 3L7 19l-4 1 1-4L16.5 3.5z" />
          </svg>
          Change Plan
        </h2>
      </div>

      <form onSubmit={handleSubmit} className="action-form">
        <div className="input-group">
          <label htmlFor="cp-sub">Subscription</label>
          <select
            id="cp-sub"
            className="input-field"
            value={subId}
            onChange={(e) => { setSubId(e.target.value); setSuccess(false); }}
            required
          >
            <option value="">Select a subscription...</option>
            {subscriptions.map((s) => (
              <option key={s.id} value={s.id}>
                {s.planName} ({s.id})
              </option>
            ))}
          </select>
        </div>

        <div className="input-group">
          <label htmlFor="cp-plan">New Plan</label>
```

```

    <select
      id="cp-plan"
      className="input-field"
      value={planId}
      onChange={(e) => { setPlanId(e.target.value); setSuccess(false); }}
      required
    >
      <option value="">Select new plan...</option>
      {plans.map((p) => (
        <option key={p.id} value={p.id}>
          {p.name} - ${parseFloat(p.price)}/{p.billingPeriod === 'MONTHLY' ? 'mo' : 'yr'}
        </option>
      ))}
    </select>
  </div>

  <div className="input-group">
    <label htmlFor="cp-pm">Payment Method ID</label>
    <input
      id="cp-pm"
      className="input-field"
      type="text"
      placeholder="e.g. pm_12345"
      value={paymentMethod}
      onChange={(e) => { setPaymentMethod(e.target.value); setSuccess(false); }}
      required
    />
  </div>

  {error && <div className="form-error">{error}</div>}
  {success && <div className="form-success">Plan changed successfully!</div>}

  <button type="submit" className="btn btn-primary" disabled={loading || !subId || !planId || !paymentMethod}>
    {loading ? 'Processing...' : 'Submit Change'}
  </button>
</form>
</div>
);
}

```

File: frontend/src/component/usageForm.jsx

```
import { useState } from 'react';
import { ingestUsage } from '../api';

export default function UsageForm({ apiKey, subscriptions, addLog }) {
  const [subId, setSubId] = useState('');
  const [quantity, setQuantity] = useState('');
  const [eventType, setEventType] = useState('API_CALL');
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState('');
  const [success, setSuccess] = useState(false);

  const handleSubmit = async (e) => {
    e.preventDefault();
    if (!subId || !quantity || !eventType) return;

    setLoading(true);
    setError('');
    setSuccess(false);

    const res = await ingestUsage(apiKey, subId, quantity, eventType);
    addLog({
      method: res.meta.method,
      url: res.meta.url,
      status: res.status,
      elapsed: res.meta.elapsed,
      body: res.data,
    });

    if (res.ok) {
      setSuccess(true);
      setQuantity('');
    } else {
      setError(res.data?.error || 'Failed to ingest usage');
    }
    setLoading(false);
  };

  return (
    <div className="panel glass">
      <div className="section-header">
        <h2 className="section-title">
          <svg className="icon" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round">
            <polyline points="22 12 18 12 15 21 9 3 6 12 2 12" />
          </svg>
          Ingest Usage
        </h2>
      </div>

      <form onSubmit={handleSubmit} className="action-form">
        <div className="input-group">
          <label htmlFor="usg-sub">Subscription</label>
          <select
            id="usg-sub"
            className="input-field"
            value={subId}
            onChange={(e) => { setSubId(e.target.value); setSuccess(false); }}
            required
          >
            <option value="">Select a subscription...</option>
            {subscriptions.map((s) => (
              <option key={s.id} value={s.id}>
                {s.planName} ({s.id})
              </option>
            ))}
          </select>
        </div>

        <div className="input-row">
          <div className="input-group" style={{ flex: 1 }}>
            <label htmlFor="usg-qty">Quantity</label>
            <input
              id="usg-qty"
              className="input-field"
              type="number"
              step="0.01"
              min="0.01"
              placeholder="0.00"
            />
          </div>
        </div>
      </form>
    </div>
  );
}
```

```
        value={quantity}
        onChange={(e) => { setQuantity(e.target.value); setSuccess(false); }}
        required
      />
    </div>

    <div className="input-group" style={{ flex: 1 }}>
      <label htmlFor="usg-evt">Event Type</label>
      <input
        id="usg-evt"
        className="input-field"
        type="text"
        placeholder="API_CALL"
        value={eventType}
        onChange={(e) => { setEventType(e.target.value); setSuccess(false); }}
        required
      />
    </div>
  </div>

  {error && <div className="form-error">{error}</div>}
  {success && <div className="form-success">Usage ingested successfully!</div>}

  <button type="submit" className="btn btn-primary" disabled={loading || !subId || !quantity || !eventType}>
    {loading ? 'Sending...' : 'Ingest Event'}
  </button>
</form>
</div>
);
}
```

File: `frontend/src/components/ApiLog.jsx`

```
import './ApiLog.css';

function statusColor(status) {
  if (status >= 200 && status < 300) return 'color-success';
  if (status >= 400 && status < 500) return 'color-warning';
  if (status >= 500) return 'color-error';
  return '';
}

export default function ApiLog({ logs, onClear }) {
  return (
    <div className="api-log glass">
      <div className="api-log-header">
        <h2 className="section-title">
          <svg className="icon" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round">
            <polyline points="4 17 10 11 4 5" />
            <line x1="12" y1="19" x2="20" y2="19" />
          </svg>
          API Request Log
        </h2>
        <div className="api-log-actions">
          <span className="log-count">{logs.length} request{logs.length !== 1 && 's'}</span>
          <button className="btn-clear" onClick={onClear} disabled={logs.length === 0}>Clear</button>
        </div>
      </div>

      <div className="api-log-body">
        {logs.length === 0 ? (
          <div className="log-empty">No API requests made yet. Action logs will appear here.</div>
        ) : (
          <div className="log-entries">
            {logs.map((log) => (
              <div key={log.id} className="log-entry">
                <div className="log-entry-meta">
                  <span className="log-time">{log.timestamp}</span>
                  <span className={`log-method ${log.method.toLowerCase()}`}>{log.method}</span>
                  <span className="log-url">{log.url}</span>
                  <span className="log-spacer" />
                  {log.idempotencyKey && (
                    <span className="log-idem" title="Idempotency Key">
                      <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2">
                        <rect x="3" y="11" width="18" height="11" rx="2" ry="2"></rect>
                        <path d="M7 11V7a5 5 0 0 1 10 0v4"></path>
                      </svg>
                      {log.idempotencyKey}
                    </span>
                  )}
                <span className={`log-status ${statusColor(log.status)}`}>{log.status}</span>
                <span className="log-elapsed">{log.elapsed}ms</span>
              </div>
              <div className="log-entry-body">
                <pre>{JSON.stringify(log.body, null, 2)}</pre>
              </div>
            </div>
          </div>
        ))}
      </div>
    </div>
  );
}
```

File: `src/main/java/com/subsflow/SubsFlowApplication.java`

```
package com.subsflow;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class SubsFlowApplication {

  public static void main(String[] args) {
    SpringApplication.run(SubsFlowApplication.class, args);
  }
}
```


File: src/main/java/com/subsflow/tenant/entity/Tenant.java

```
package com.subsflow.tenant.entity;

import jakarta.persistence.*;
import lombok.Getter;
import lombok.Setter;
import java.time.OffsetDateTime;

@Entity
@Table(name = "tenant")
@Getter
@Setter
public class Tenant {

    @Id
    private String id;

    @Column(nullable = false)
    private String name;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private TenantStatus status;

    @Column(name = "api_key", nullable = false, unique = true)
    private String apiKey;

    @Column(name = "created_at", nullable = false, updatable = false)
    private OffsetDateTime createdAt = OffsetDateTime.now();

    @Column(name = "updated_at", nullable = false)
    private OffsetDateTime updatedAt = OffsetDateTime.now();

    @PreUpdate
    protected void onUpdate() {
        updatedAt = OffsetDateTime.now();
    }
}
```

File: src/main/java/com/subsflow/tenant/entity/TenantStatus.java

```
package com.subsflow.tenant.entity;

public enum TenantStatus {
    ACTIVE,
    SUSPENDED
}
```

File: src/main/java/com/subsflow/tenant/repository/TenantRepository.java

```
package com.subsflow.tenant.repository;

import com.subsflow.tenant.entity.Tenant;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.Optional;

public interface TenantRepository extends JpaRepository<Tenant, String> {
    Optional<Tenant> findByApiKey(String apiKey);
}
```

File: src/main/java/com/subsflow/tenant/controller/TenantController.java

```
package com.subsflow.tenant.controller;

import com.subsflow.tenant.entity.Tenant;
import com.subsflow.tenant.entity.TenantStatus;
import com.subsflow.tenant.repository.TenantRepository;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.Map;
import java.util.UUID;

@RestController
@RequestMapping("/api/v1/tenants")
public class TenantController {

    private final TenantRepository tenantRepository;

    public TenantController(TenantRepository tenantRepository) {
        this.tenantRepository = tenantRepository;
    }

    @PostMapping
    public ResponseEntity<Tenant> onboardTenant(@RequestBody OnboardTenantRequest request) {
        if (request.getName() == null || request.getName().trim().isEmpty()) {
            return ResponseEntity.badRequest().build();
        }

        Tenant tenant = new Tenant();
        tenant.setId("t_" + UUID.randomUUID().toString().substring(0, 8));
        tenant.setName(request.getName());
        tenant.setStatus(TenantStatus.ACTIVE);
        tenant.setApiKey("sk_" + UUID.randomUUID().toString().replace("-", ""));

        Tenant saved = tenantRepository.save(tenant);
        return ResponseEntity.ok(saved);
    }

    @PostMapping("/login")
    public ResponseEntity<?> loginTenant(@RequestBody LoginTenantRequest request) {
        if (request.getApiKey() == null || request.getApiKey().trim().isEmpty()) {
            return ResponseEntity.badRequest().body(Map.of("error", "API key is required"));
        }

        return tenantRepository.findByApiKey(request.getApiKey())
            .map(tenant -> ResponseEntity.ok(Map.of(
                "id", tenant.getId(),
                "name", tenant.getName(),
                "status", tenant.getStatus().name(),
                "apiKey", tenant.getApiKey()
            )))
            .orElseGet(() -> ResponseEntity.status(404).body(Map.of("error", "Tenant not found")));
    }

    public static class OnboardTenantRequest {
        private String name;

        public String getName() {
            return name;
        }

        public void setName(String name) {
            this.name = name;
        }
    }

    public static class LoginTenantRequest {
        private String apiKey;

        public String getApiKey() {
            return apiKey;
        }

        public void setApiKey(String apiKey) {
            this.apiKey = apiKey;
        }
    }
}
```

File: `src/main/java/com/subsflow/subscription/entity/BillingPeriod.java`

```
package com.subsflow.subscription.entity;

public enum BillingPeriod {
    MONTHLY,
    YEARLY
}
```

File: `src/main/java/com/subsflow/subscription/entity/BillingType.java`

```
package com.subsflow.subscription.entity;

public enum BillingType {
    FIXED,
    USAGE_BASED,
    TIERED
}
```

File: `src/main/java/com/subsflow/subscription/entity/BillingPlan.java`

```
package com.subsflow.subscription.entity;

import com.subsflow.common.entity.AbstractTenantEntity;
import jakarta.persistence.*;
import lombok.Getter;
import lombok.Setter;

import java.math.BigDecimal;
import java.time.OffsetDateTime;

@Entity
@Table(name = "billing_plan")
@Getter
@Setter
public class BillingPlan extends AbstractTenantEntity {

    @Id
    private String id;

    @Column(nullable = false)
    private String name;

    @Enumerated(EnumType.STRING)
    @Column(name = "billing_type", nullable = false)
    private BillingType billingType;

    @Column(nullable = false, precision = 20, scale = 4)
    private BigDecimal price;

    @Enumerated(EnumType.STRING)
    @Column(name = "billing_period", nullable = false)
    private BillingPeriod billingPeriod;

    @Version
    @Column(nullable = false)
    private Long version;

    @Column(name = "created_at", nullable = false, updatable = false)
    private OffsetDateTime createdAt = OffsetDateTime.now();

    @Column(name = "updated_at", nullable = false)
    private OffsetDateTime updatedAt = OffsetDateTime.now();

    @PreUpdate
    protected void onUpdate() {
        updatedAt = OffsetDateTime.now();
    }
}
```

File: `src/main/java/com/subsflow/subscription/entity/SubscriptionStatus.java`

```
package com.subsflow.subscription.entity;

public enum SubscriptionStatus {
    TRIAL,
    ACTIVE,
    PAST_DUE,
    SUSPENDED,
    CANCELLED
}
```

File: `src/main/java/com/subsflow/subscription/entity/Subscription.java`

```
package com.subsflow.subscription.entity;

import com.subsflow.common.entity.AbstractTenantEntity;
import jakarta.persistence.*;
import lombok.Getter;
import lombok.Setter;

import java.time.OffsetDateTime;

@Entity
@Table(name = "subscription")
@Getter
@Setter
public class Subscription extends AbstractTenantEntity {

    @Id
    private String id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "plan_id", nullable = false)
    private BillingPlan plan;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private SubscriptionStatus status;

    @Column(name = "current_period_start", nullable = false)
    private OffsetDateTime currentPeriodStart;

    @Column(name = "current_period_end", nullable = false)
    private OffsetDateTime currentPeriodEnd;

    @Version
    @Column(nullable = false)
    private Long version;

    @Column(name = "created_at", nullable = false, updatable = false)
    private OffsetDateTime createdAt = OffsetDateTime.now();

    @Column(name = "updated_at", nullable = false)
    private OffsetDateTime updatedAt = OffsetDateTime.now();

    @PreUpdate
    protected void onUpdate() {
        updatedAt = OffsetDateTime.now();
    }
}
```

File: `src/main/java/com/subsflow/subscription/repository/BillingPlanRepository.java`

```
package com.subsflow.subscription.repository;

import com.subsflow.subscription.entity.BillingPlan;
import org.springframework.data.jpa.repository.JpaRepository;

public interface BillingPlanRepository extends JpaRepository<BillingPlan, String> {
}
```

File: `src/main/java/com/subsflow/subscription/repository/SubscriptionRepository.java`

```
package com.subsflow.subscription.repository;

import com.subsflow.subscription.entity.Subscription;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;

import java.time.OffsetDateTime;
import java.util.List;

public interface SubscriptionRepository extends JpaRepository<Subscription, String> {

    @Query(value = "SELECT * FROM subscription WHERE status IN ('ACTIVE', 'TRIAL') AND current_period_end <= :now ORDER BY current_period_end ASC LIMIT 100 FOR UPDATE SKIP LOCKED", nativeQuery = true)
    List<Subscription> findDueSubscriptions(OffsetDateTime now);
}
```

File: `src/main/java/com/subsflow/subscription/service/SubscriptionStatusTransition.java`

```
package com.subsflow.subscription.service;

import com.subsflow.subscription.entity.SubscriptionStatus;

public final class SubscriptionStatusTransition {

    private SubscriptionStatusTransition() {}

    public static void requireTransition(SubscriptionStatus current, SubscriptionStatus target, String operation) {
        if (current == SubscriptionStatus.CANCELLED && target == SubscriptionStatus.ACTIVE) {
            throw new IllegalStateException("Subscription reactivation is not allowed after cancellation for " + operation);
        }
        if (current == SubscriptionStatus.SUSPENDED && target == SubscriptionStatus.ACTIVE) {
            return;
        }
        if (current == SubscriptionStatus.PAST_DUE && target == SubscriptionStatus.ACTIVE) {
            return;
        }
        if (current == SubscriptionStatus.ACTIVE && target == SubscriptionStatus.PAST_DUE) {
            return;
        }
        if (current == SubscriptionStatus.ACTIVE && target == SubscriptionStatus.SUSPENDED) {
            return;
        }
        if (current == SubscriptionStatus.PAST_DUE && target == SubscriptionStatus.SUSPENDED) {
            return;
        }
        if (current == SubscriptionStatus.ACTIVE && target == SubscriptionStatus.CANCELLED) {
            return;
        }
        if (current == SubscriptionStatus.PAST_DUE && target == SubscriptionStatus.CANCELLED) {
            return;
        }
    }
}
```

File: `src/main/java/com/subsflow/subscription/service/SubscriptionService.java`

```
package com.subsflow.subscription.service;

import com.subsflow.subscription.entity.Subscription;
import java.math.BigDecimal;

public interface SubscriptionService {

    /**
     * Changes a subscription's plan mid-cycle, calculating proration, generating invoices,
     * executing charges via the payment service, and updating status under optimistic locks.
     */
    Subscription changePlan(String subscriptionId, String newPlanId, String paymentMethodId, String idempotencyKey);

    /**
     * Creates a new subscription for a given plan.
     */
    Subscription createSubscription(String planId);

    /**
     * Cancels an existing active or past-due subscription.
     */
    Subscription cancelSubscription(String subscriptionId);

    /**
     * Records a usage event for the current cycle.
     */
    void ingestUsage(String subscriptionId, BigDecimal quantity, String eventType);
}
```

File: src/main/java/com/subsflow/subscription/service/SubscriptionServiceImpl.java

```
package com.subsflow.subscription.service;

import com.fasterxml.jackson.databind.ObjectMapper;
import com.subsflow.billing.entity.*;
import com.subsflow.billing.repository.InvoiceLineItemRepository;
import com.subsflow.billing.repository.InvoiceRepository;
import com.subsflow.billing.repository.UsageEventRepository;
import com.subsflow.common.entity.OutboxEvent;
import com.subsflow.common.entity.OutboxEventStatus;
import com.subsflow.common.repository.OutboxEventRepository;
import com.subsflow.payment.entity.PaymentStatus;
import com.subsflow.payment.entity.PaymentTransaction;
import com.subsflow.payment.service.PaymentService;
import com.subsflow.subscription.entity.BillingPlan;
import com.subsflow.subscription.entity.Subscription;
import com.subsflow.subscription.entity.SubscriptionStatus;
import com.subsflow.subscription.repository.BillingPlanRepository;
import com.subsflow.subscription.repository.SubscriptionRepository;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.time.Duration;
import java.time.OffsetDateTime;
import java.util.Map;
import java.util.UUID;

@Service
public class SubscriptionServiceImpl implements SubscriptionService {

    private static final Logger log = LoggerFactory.getLogger(SubscriptionServiceImpl.class);

    private final SubscriptionRepository subscriptionRepository;
    private final BillingPlanRepository billingPlanRepository;
    private final UsageEventRepository usageEventRepository;
    private final InvoiceRepository invoiceRepository;
    private final InvoiceLineItemRepository invoiceLineItemRepository;
    private final PaymentService paymentService;
    private final OutboxEventRepository outboxEventRepository;
    private final ObjectMapper objectMapper;

    public SubscriptionServiceImpl(SubscriptionRepository subscriptionRepository,
                                BillingPlanRepository billingPlanRepository,
                                UsageEventRepository usageEventRepository,
                                InvoiceRepository invoiceRepository,
                                InvoiceLineItemRepository invoiceLineItemRepository,
                                PaymentService paymentService,
                                OutboxEventRepository outboxEventRepository,
                                ObjectMapper objectMapper) {
        this.subscriptionRepository = subscriptionRepository;
        this.billingPlanRepository = billingPlanRepository;
        this.usageEventRepository = usageEventRepository;
        this.invoiceRepository = invoiceRepository;
        this.invoiceLineItemRepository = invoiceLineItemRepository;
        this.paymentService = paymentService;
        this.outboxEventRepository = outboxEventRepository;
        this.objectMapper = objectMapper;
    }

    @Override
    @Transactional
    public Subscription changePlan(String subscriptionId, String newPlanId, String paymentMethodId, String idempotencyKey) {
        log.info("Processing plan change for subscription: {} to plan: {}", subscriptionId, newPlanId);

        Subscription subscription = subscriptionRepository.findById(subscriptionId)
            .orElseThrow(() -> new IllegalArgumentException("Subscription not found: " + subscriptionId));

        BillingPlan newPlan = billingPlanRepository.findById(newPlanId)
            .orElseThrow(() -> new IllegalArgumentException("Billing plan not found: " + newPlanId));

        BillingPlan oldPlan = subscription.getPlan();
        if (oldPlan.getId().equals(newPlanId)) {
            log.info("Subscription is already on plan: {}. Bypassing change.", newPlanId);
            return subscription;
        }
    }
}
```

```

}

OffsetDateTime now = OffsetDateTime.now();
OffsetDateTime start = subscription.getCurrentPeriodStart();
OffsetDateTime end = subscription.getCurrentPeriodEnd();

// Calculate ratios for proration
long totalSeconds = Duration.between(start, end).toSeconds();
long remainingSeconds = Math.max(0, Duration.between(now, end).toSeconds());
if (totalSeconds <= 0) {
    totalSeconds = 1;
}

BigDecimal ratio = BigDecimal.valueOf(remainingSeconds)
    .divide(BigDecimal.valueOf(totalSeconds), 8, RoundingMode.HALF_UP);

BigDecimal credit = oldPlan.getPrice().multiply(ratio).setScale(4, RoundingMode.HALF_UP);
BigDecimal cost = newPlan.getPrice().multiply(ratio).setScale(4, RoundingMode.HALF_UP);
BigDecimal netAmount = cost.subtract(credit);

// Create Draft Invoice
Invoice invoice = new Invoice();
invoice.setId("inv_" + UUID.randomUUID().toString().substring(0, 8));
invoice.setSubscription(subscription);
invoice.setStatus(InvoiceStatus.DRAFT);
invoice.setDueDate(now.plusDays(7));

// If netAmount is negative, we credit it to 0 for simplicity
BigDecimal chargeAmount = netAmount.compareTo(BigDecimal.ZERO) > 0 ? netAmount : BigDecimal.ZERO;
invoice.setAmount(chargeAmount);
invoiceRepository.save(invoice);

// Line Item 1: Old Plan Credit
InvoiceLineItem refundItem = new InvoiceLineItem();
refundItem.setId("ili_" + UUID.randomUUID().toString().substring(0, 8));
refundItem.setInvoice(invoice);
refundItem.setPlan(oldPlan);
refundItem.setType(InvoiceLineItemType.PRORATION);
refundItem.setAmount(credit.negate());
refundItem.setDescription("Unused time credit on " + oldPlan.getName());
invoiceLineItemRepository.save(refundItem);

// Line Item 2: New Plan Charge
InvoiceLineItem chargeItem = new InvoiceLineItem();
chargeItem.setId("ili_" + UUID.randomUUID().toString().substring(0, 8));
chargeItem.setInvoice(invoice);
chargeItem.setPlan(newPlan);
chargeItem.setType(InvoiceLineItemType.PRORATION);
chargeItem.setAmount(cost);
chargeItem.setDescription("Remaining time charge on " + newPlan.getName());
invoiceLineItemRepository.save(chargeItem);

invoice.getLineItems().add(refundItem);
invoice.getLineItems().add(chargeItem);

boolean paymentSuccess = true;
if (chargeAmount.compareTo(BigDecimal.ZERO) > 0) {
    PaymentTransaction transaction = paymentService.processPayment(subscription, invoice, paymentMethodId,
        idempotencyKey);
    if (transaction.getStatus() != PaymentStatus.SUCCESS) {
        paymentSuccess = false;
    }
} else {
    // Amount is 0 or negative: instantly mark invoice as PAID
    invoice.setStatus(InvoiceStatus.PAID);
    invoiceRepository.save(invoice);
}

// Apply changes to Subscription
subscription.setPlan(newPlan);
SubscriptionStatus targetStatus = paymentSuccess ? SubscriptionStatus.ACTIVE : SubscriptionStatus.PAST_DUE;
SubscriptionStatusTransition.requireTransition(subscription.getStatus(), targetStatus, "plan change");
subscription.setStatus(targetStatus);
Subscription updatedSubscription = subscriptionRepository.save(subscription);

// Enqueue Outbox Event
writeOutboxEvent("subscription.changed", Map.of(
    "subscriptionId", subscription.getId(),

```

```

        "oldPlanId", oldPlan.getId(),
        "newPlanId", newPlan.getId(),
        "status", subscription.getStatus().name(),
        "prorationAmount", chargeAmount
    ));

    return updatedSubscription;
}

@Override
@Transactional
public void ingestUsage(String subscriptionId, BigDecimal quantity, String eventType) {
    log.info("Ingesting usage event for subscription: {}, quantity: {}, type: {}", subscriptionId, quantity, eventType);

    Subscription subscription = subscriptionRepository.findById(subscriptionId)
        .orElseThrow(() -> new IllegalArgumentException("Subscription not found: " + subscriptionId));

    UsageEvent event = new UsageEvent();
    event.setId("use_" + UUID.randomUUID().toString().substring(0, 8));
    event.setSubscription(subscription);
    event.setQuantity(quantity.setScale(4, RoundingMode.HALF_UP));
    event.setEventType(eventType);
    event.setTimestamp(OffsetDateTime.now());
    usageEventRepository.save(event);
}

@Override
@Transactional
public Subscription createSubscription(String planId) {
    log.info("Creating subscription for plan: {}", planId);
    BillingPlan plan = billingPlanRepository.findById(planId)
        .orElseThrow(() -> new IllegalArgumentException("Billing plan not found: " + planId));

    OffsetDateTime now = OffsetDateTime.now();
    OffsetDateTime periodEnd = plan.getBillingPeriod() == com.subsflow.subscription.entity.BillingPeriod.YEARLY
        ? now.plusYears(1) : now.plusMonths(1);

    Subscription subscription = new Subscription();
    subscription.setId("sub_" + UUID.randomUUID().toString().substring(0, 8));
    subscription.setPlan(plan);
    subscription.setStatus(SubscriptionStatus.ACTIVE);
    subscription.setCurrentPeriodStart(now);
    subscription.setCurrentPeriodEnd(periodEnd);
    subscription.setVersion(0L);

    Subscription saved = subscriptionRepository.save(subscription);

    writeOutboxEvent("subscription.created", Map.of(
        "subscriptionId", saved.getId(),
        "planId", plan.getId(),
        "status", saved.getStatus().name()
    ));

    return saved;
}

@Override
@Transactional
public Subscription cancelSubscription(String subscriptionId) {
    log.info("Cancelling subscription: {}", subscriptionId);
    Subscription subscription = subscriptionRepository.findById(subscriptionId)
        .orElseThrow(() -> new IllegalArgumentException("Subscription not found: " + subscriptionId));

    SubscriptionStatusTransition.requireTransition(subscription.getStatus(), SubscriptionStatus.CANCELLED,
"cancellation");
    subscription.setStatus(SubscriptionStatus.CANCELLED);
    Subscription saved = subscriptionRepository.save(subscription);

    writeOutboxEvent("subscription.cancelled", Map.of(
        "subscriptionId", saved.getId(),
        "status", saved.getStatus().name()
    ));

    return saved;
}

private void writeOutboxEvent(String eventType, Object payload) {
    try {

```



```
        String jsonPayload = objectMapper.writeValueAsString(payload);
        OutboxEvent event = new OutboxEvent();
        event.setId("evt_" + UUID.randomUUID().toString().substring(0, 8));
        event.setEventType(eventType);
        event.setPayload(jsonPayload);
        event.setStatus(OutboxEventStatus.PENDING);
        outboxEventRepository.save(event);
    } catch (Exception e) {
        log.error("Failed to write outbox event", e);
    }
}
```

File: src/main/java/com/subsflow/subscription/controller/SubscriptionController.java

```
package com.subsflow.subscription.controller;

import com.fasterxml.jackson.databind.ObjectMapper;
import com.subsflow.common.exception.IdempotencyConflictException;
import com.subsflow.common.exception.IdempotencyKeyUsedException;
import com.subsflow.payment.service.IdempotencyService;
import com.subsflow.subscription.entity.BillingPlan;
import com.subsflow.subscription.entity.Subscription;
import com.subsflow.subscription.repository.BillingPlanRepository;
import com.subsflow.subscription.repository.SubscriptionRepository;
import com.subsflow.subscription.service.SubscriptionService;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.orm.ObjectOptimisticLockingFailureException;
import org.springframework.web.bind.annotation.*;

import java.math.BigDecimal;
import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;
import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;

@RestController
@RequestMapping("/api/v1/subscriptions")
public class SubscriptionController {

    private final SubscriptionService subscriptionService;
    private final IdempotencyService idempotencyService;
    private final ObjectMapper objectMapper;
    private final SubscriptionRepository subscriptionRepository;
    private final BillingPlanRepository billingPlanRepository;

    public SubscriptionController(SubscriptionService subscriptionService,
                                IdempotencyService idempotencyService,
                                ObjectMapper objectMapper,
                                SubscriptionRepository subscriptionRepository,
                                BillingPlanRepository billingPlanRepository) {
        this.subscriptionService = subscriptionService;
        this.idempotencyService = idempotencyService;
        this.objectMapper = objectMapper;
        this.subscriptionRepository = subscriptionRepository;
        this.billingPlanRepository = billingPlanRepository;
    }

    @GetMapping
    public ResponseEntity<List<SubscriptionSummaryResponse>> listSubscriptions() {
        List<SubscriptionSummaryResponse> subscriptions = subscriptionRepository.findAll().stream()
            .map(SubscriptionSummaryResponse::from)
            .collect(Collectors.toList());
        return ResponseEntity.ok(subscriptions);
    }

    @GetMapping("/plans")
    public ResponseEntity<List<BillingPlanResponse>> listPlans() {
        List<BillingPlanResponse> plans = billingPlanRepository.findAll().stream()
            .map(BillingPlanResponse::from)
            .collect(Collectors.toList());
        return ResponseEntity.ok(plans);
    }

    @PostMapping
    public ResponseEntity<?> createSubscription(@RequestBody CreateSubscriptionRequest request) {
        if (request.getPlanId() == null || request.getPlanId().trim().isEmpty()) {
            return ResponseEntity.badRequest().body(Map.of("error", "planId is required"));
        }
        try {
            Subscription subscription = subscriptionService.createSubscription(request.getPlanId());
            return ResponseEntity.ok(SubscriptionSummaryResponse.from(subscription));
        } catch (Exception e) {
            return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(Map.of("error", e.getMessage()));
        }
    }

    @PostMapping("/{id}/cancel")
    public ResponseEntity<?> cancelSubscription(@PathVariable("id") String subscriptionId) {
        try {
```

```

        Subscription subscription = subscriptionService.cancelSubscription(subscriptionId);
        return ResponseEntity.ok(SubscriptionSummaryResponse.from(subscription));
    } catch (Exception e) {
        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(Map.of("error", e.getMessage()));
    }
}

@PostMapping("/{id}/change-plan")
public ResponseEntity<?> changePlan(
    @PathVariable("id") String subscriptionId,
    @RequestHeader(value = "Idempotency-Key") String idempotencyKey,
    @RequestBody ChangePlanRequest request) {

    if (request.getNewPlanId() == null || request.getPaymentMethodId() == null) {
        return ResponseEntity.badRequest().body(Map.of("error", "Missing required fields newPlanId or paymentMethodId"));
    }

    if (idempotencyKey == null || idempotencyKey.trim().isEmpty()) {
        return ResponseEntity.badRequest().body(Map.of("error", "Idempotency-Key header is required"));
    }

    // Calculate SHA-256 Hash of the input parameters to detect key reuse misuse
    String requestHash = calculateHash(request.getNewPlanId(), request.getPaymentMethodId());

    try {
        idempotencyService.startOperation(idempotencyKey, requestHash);
    } catch (IdempotencyConflictException e) {
        return ResponseEntity.status(HttpStatus.CONFLICT).body(Map.of("error", e.getMessage()));
    } catch (IdempotencyKeyUsedException e) {
        // Already completed request - return cached response
        try {
            Object cachedResponse = objectMapper.readValue(e.getResponsePayload(), Object.class);
            return ResponseEntity.ok()
                .header("X-Cache-Lookup", "HIT")
                .body(cachedResponse);
        } catch (Exception ex) {
            return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(Map.of("error", "Failed to deserialize
cached response"));
        }
    }

    // Execute the change plan business operation
    try {
        Subscription updatedSub = subscriptionService.changePlan(subscriptionId, request.getNewPlanId(),
request.getPaymentMethodId(), idempotencyKey);
        String jsonResponse = objectMapper.writeValueAsString(updatedSub);
        idempotencyService.completeOperation(idempotencyKey, jsonResponse);
        return ResponseEntity.ok(updatedSub);
    } catch (ObjectOptimisticLockingFailureException e) {
        return ResponseEntity.status(HttpStatus.CONFLICT).body(Map.of("error", "Subscription was updated concurrently,
please retry"));
    } catch (IllegalStateException e) {
        return ResponseEntity.status(HttpStatus.CONFLICT).body(Map.of("error", e.getMessage()));
    } catch (Exception e) {
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(Map.of("error", e.getMessage()));
    }
}

@PostMapping("/{id}/usage")
public ResponseEntity<?> ingestUsage(
    @PathVariable("id") String subscriptionId,
    @RequestBody IngestUsageRequest request) {

    if (request.getQuantity() == null || request.getQuantity().compareTo(BigDecimal.ZERO) <= 0 || request.getEventType()
== null) {
        return ResponseEntity.badRequest().body(Map.of("error", "Invalid quantity or eventType"));
    }

    try {
        subscriptionService.ingestUsage(subscriptionId, request.getQuantity(), request.getEventType());
        return ResponseEntity.ok(Map.of("status", "success"));
    } catch (Exception e) {
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(Map.of("error", e.getMessage()));
    }
}

private String calculateHash(String newPlanId, String paymentMethodId) {
    try {

```

```

        String raw = newPlanId + "|" + paymentMethodId;
        MessageDigest digest = MessageDigest.getInstance("SHA-256");
        byte[] hash = digest.digest(raw.getBytes(StandardCharsets.UTF_8));
        StringBuilder hexString = new StringBuilder();
        for (byte b : hash) {
            String hex = Integer.toHexString(0xff & b);
            if (hex.length() == 1) hexString.append('0');
            hexString.append(hex);
        }
        return hexString.toString();
    } catch (Exception e) {
        throw new RuntimeException("Hashing failed", e);
    }
}

public static class SubscriptionSummaryResponse {
    private String id;
    private String status;
    private String planId;
    private String planName;
    private String currentPeriodStart;
    private String currentPeriodEnd;

    public static SubscriptionSummaryResponse from(Subscription subscription) {
        SubscriptionSummaryResponse dto = new SubscriptionSummaryResponse();
        dto.id = subscription.getId();
        dto.status = subscription.getStatus().name();
        BillingPlan plan = subscription.getPlan();
        if (plan != null) {
            dto.planId = plan.getId();
            dto.planName = plan.getName();
        }
        dto.currentPeriodStart = subscription.getCurrentPeriodStart() == null ? null :
subscription.getCurrentPeriodStart().toString();
        dto.currentPeriodEnd = subscription.getCurrentPeriodEnd() == null ? null :
subscription.getCurrentPeriodEnd().toString();
        return dto;
    }

    public String getId() { return id; }
    public String getStatus() { return status; }
    public String getPlanId() { return planId; }
    public String getPlanName() { return planName; }
    public String getCurrentPeriodStart() { return currentPeriodStart; }
    public String getCurrentPeriodEnd() { return currentPeriodEnd; }
}

public static class BillingPlanResponse {
    private String id;
    private String name;
    private String billingType;
    private String billingPeriod;
    private String price;

    public static BillingPlanResponse from(BillingPlan plan) {
        BillingPlanResponse dto = new BillingPlanResponse();
        dto.id = plan.getId();
        dto.name = plan.getName();
        dto.billingType = plan.getBillingType().name();
        dto.billingPeriod = plan.getBillingPeriod().name();
        dto.price = plan.getPrice().toPlainString();
        return dto;
    }

    public String getId() { return id; }
    public String getName() { return name; }
    public String getBillingType() { return billingType; }
    public String getBillingPeriod() { return billingPeriod; }
    public String getPrice() { return price; }
}

public static class ChangePlanRequest {
    private String newPlanId;
    private String paymentMethodId;

    public String getNewPlanId() {
        return newPlanId;
    }
}

```

```

    public void setNewPlanId(String newPlanId) {
        this.newPlanId = newPlanId;
    }

    public String getPaymentMethodId() {
        return paymentMethodId;
    }

    public void setPaymentMethodId(String paymentMethodId) {
        this.paymentMethodId = paymentMethodId;
    }
}

public static class IngestUsageRequest {
    private BigDecimal quantity;
    private String eventType;

    public BigDecimal getQuantity() {
        return quantity;
    }

    public void setQuantity(BigDecimal quantity) {
        this.quantity = quantity;
    }

    public String getEventType() {
        return eventType;
    }

    public void setEventType(String eventType) {
        this.eventType = eventType;
    }
}

public static class CreateSubscriptionRequest {
    private String planId;

    public String getPlanId() {
        return planId;
    }

    public void setPlanId(String planId) {
        this.planId = planId;
    }
}
}

```

File: `src/main/java/com/subsflow/billing/entity/InvoiceStatus.java`

```

package com.subsflow.billing.entity;

public enum InvoiceStatus {
    DRAFT,
    UNPAID,
    PAID,
    VOID
}

```

File: `src/main/java/com/subsflow/billing/entity/InvoiceLineItemType.java`

```

package com.subsflow.billing.entity;

public enum InvoiceLineItemType {
    FLAT,
    USAGE,
    PRORATION
}

```

File: `src/main/java/com/subsflow/billing/entity/InvoiceLineItem.java`

```
package com.subsflow.billing.entity;

import com.subsflow.common.entity.AbstractTenantEntity;
import com.subsflow.subscription.entity.BillingPlan;
import jakarta.persistence.*;
import lombok.Getter;
import lombok.Setter;

import java.math.BigDecimal;
import java.time.OffsetDateTime;

@Entity
@Table(name = "invoice_line_item")
@Getter
@Setter
public class InvoiceLineItem extends AbstractTenantEntity {

    @Id
    private String id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "invoice_id", nullable = false)
    private Invoice invoice;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "plan_id")
    private BillingPlan plan;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private InvoiceLineItemType type;

    @Column(nullable = false, precision = 20, scale = 4)
    private BigDecimal amount;

    @Column
    private String description;

    @Column(name = "created_at", nullable = false, updatable = false)
    private OffsetDateTime createdAt = OffsetDateTime.now();
}
```

File: src/main/java/com/subsflow/billing/entity/invoice.java

```
package com.subsflow.billing.entity;

import com.subsflow.common.entity.AbstractTenantEntity;
import com.subsflow.subscription.entity.Subscription;
import jakarta.persistence.*;
import lombok.Getter;
import lombok.Setter;

import java.math.BigDecimal;
import java.time.OffsetDateTime;
import java.util.ArrayList;
import java.util.List;

@Entity
@Table(name = "invoice")
@Getter
@Setter
public class Invoice extends AbstractTenantEntity {

    @Id
    private String id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "subscription_id", nullable = false)
    private Subscription subscription;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private InvoiceStatus status;

    @Column(nullable = false, precision = 20, scale = 4)
    private BigDecimal amount;

    @Column(name = "due_date", nullable = false)
    private OffsetDateTime dueDate;

    @OneToMany(mappedBy = "invoice", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<InvoiceLineItem> lineItems = new ArrayList<>();

    @Column(name = "created_at", nullable = false, updatable = false)
    private OffsetDateTime createdAt = OffsetDateTime.now();

    @Column(name = "updated_at", nullable = false)
    private OffsetDateTime updatedAt = OffsetDateTime.now();

    @PreUpdate
    protected void onUpdate() {
        updatedAt = OffsetDateTime.now();
    }
}
```

File: [src/main/java/com/subsflow/billing/entity/UsageEvent.java](#)

```
package com.subsflow.billing.entity;

import com.subsflow.common.entity.AbstractTenantEntity;
import com.subsflow.subscription.entity.Subscription;
import jakarta.persistence.*;
import lombok.Getter;
import lombok.Setter;

import java.math.BigDecimal;
import java.time.OffsetDateTime;

@Entity
@Table(name = "usage_event")
@Getter
@Setter
public class UsageEvent extends AbstractTenantEntity {

    @Id
    private String id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "subscription_id", nullable = false)
    private Subscription subscription;

    @Column(nullable = false, precision = 20, scale = 4)
    private BigDecimal quantity;

    @Column(name = "event_type", nullable = false)
    private String eventType;

    @Column(nullable = false)
    private OffsetDateTime timestamp;
}
```

File: [src/main/java/com/subsflow/billing/repository/InvoiceLineItemRepository.java](#)

```
package com.subsflow.billing.repository;

import com.subsflow.billing.entity.InvoiceLineItem;
import org.springframework.data.jpa.repository.JpaRepository;

public interface InvoiceLineItemRepository extends JpaRepository<InvoiceLineItem, String> {
}
```

File: [src/main/java/com/subsflow/billing/repository/InvoiceRepository.java](#)

```
package com.subsflow.billing.repository;

import com.subsflow.billing.entity.Invoice;
import org.springframework.data.jpa.repository.JpaRepository;

public interface InvoiceRepository extends JpaRepository<Invoice, String> {
}
```

File: [src/main/java/com/subsflow/billing/repository/UsageEventRepository.java](#)

```
package com.subsflow.billing.repository;

import com.subsflow.billing.entity.UsageEvent;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;

import java.math.BigDecimal;
import java.time.OffsetDateTime;

public interface UsageEventRepository extends JpaRepository<UsageEvent, String> {

    @Query("SELECT COALESCE(SUM(u.quantity), 0) FROM UsageEvent u WHERE u.subscription.id = :subscriptionId AND u.timestamp >= :periodStart AND u.timestamp < :periodEnd")
    BigDecimal sumUsageForPeriod(@Param("subscriptionId") String subscriptionId, @Param("periodStart") OffsetDateTime periodStart, @Param("periodEnd") OffsetDateTime periodEnd);
}
```


File: src/main/java/com/subsflow/billing/service/BillingCycleService.java

```
package com.subsflow.billing.service;

public interface BillingCycleService {
    /**
     * Bills all active subscriptions whose current billing period has ended.
     * For each due subscription:
     * 1. Aggregates usage events for usage-based plans.
     * 2. Generates the cycle invoice with flat fee + usage line items.
     * 3. Attempts immediate payment.
     * 4. If successful, extends the subscription into the next billing period.
     * 5. If failed, marks the subscription as PAST_DUE (dunning takes over).
     */
    void billDueSubscriptions(String tenantId);
}
```

File: src/main/java/com/subsflow/billing/service/BillingCycleServiceImpl.java

```
package com.subsflow.billing.service;

import com.fasterxml.jackson.databind.ObjectMapper;
import com.subsflow.billing.entity.*;
import com.subsflow.billing.repository.InvoiceLineItemRepository;
import com.subsflow.billing.repository.InvoiceRepository;
import com.subsflow.billing.repository.UsageEventRepository;
import com.subsflow.common.entity.OutboxEvent;
import com.subsflow.common.entity.OutboxEventStatus;
import com.subsflow.common.repository.OutboxEventRepository;
import com.subsflow.payment.entity.PaymentStatus;
import com.subsflow.payment.entity.PaymentTransaction;
import com.subsflow.payment.service.PaymentService;
import com.subsflow.subscription.entity.BillingPlan;
import com.subsflow.subscription.entity.BillingType;
import com.subsflow.subscription.entity.Subscription;
import com.subsflow.subscription.entity.SubscriptionStatus;
import com.subsflow.subscription.repository.SubscriptionRepository;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.time.OffsetDateTime;
import java.util.List;
import java.util.Map;
import java.util.UUID;

@Service
public class BillingCycleServiceImpl implements BillingCycleService {

    private static final Logger log = LoggerFactory.getLogger(BillingCycleServiceImpl.class);

    private final SubscriptionRepository subscriptionRepository;
    private final UsageEventRepository usageEventRepository;
    private final InvoiceRepository invoiceRepository;
    private final InvoiceLineItemRepository invoiceLineItemRepository;
    private final PaymentService paymentService;
    private final OutboxEventRepository outboxEventRepository;
    private final ObjectMapper objectMapper;

    public BillingCycleServiceImpl(SubscriptionRepository subscriptionRepository,
                                UsageEventRepository usageEventRepository,
                                InvoiceRepository invoiceRepository,
                                InvoiceLineItemRepository invoiceLineItemRepository,
                                PaymentService paymentService,
                                OutboxEventRepository outboxEventRepository,
                                ObjectMapper objectMapper) {
        this.subscriptionRepository = subscriptionRepository;
        this.usageEventRepository = usageEventRepository;
        this.invoiceRepository = invoiceRepository;
        this.invoiceLineItemRepository = invoiceLineItemRepository;
        this.paymentService = paymentService;
        this.outboxEventRepository = outboxEventRepository;
        this.objectMapper = objectMapper;
    }

    @Override
    @Transactional
    public void billDueSubscriptions(String tenantId) {
        OffsetDateTime now = OffsetDateTime.now();
        List<Subscription> dueSubscriptions = subscriptionRepository.findDueSubscriptions(now);

        if (dueSubscriptions.isEmpty()) {
            return;
        }

        log.info("Found {} due subscriptions for tenant: {}", dueSubscriptions.size(), tenantId);

        for (Subscription subscription : dueSubscriptions) {
            try {
                billSubscription(subscription);
            } catch (Exception e) {
                log.error("Error billing subscription: {}", subscription.getId(), e);
            }
        }
    }
}
```

```

    }
}

private void billSubscription(Subscription subscription) {
    BillingPlan plan = subscription.getPlan();
    OffsetDateTime periodStart = subscription.getCurrentPeriodStart();
    OffsetDateTime periodEnd = subscription.getCurrentPeriodEnd();

    log.info("Billing subscription: {} on plan: {} for period {} to {}",
        subscription.getId(), plan.getName(), periodStart, periodEnd);

    // Calculate total invoice amount
    BigDecimal totalAmount = BigDecimal.ZERO;

    // Create draft invoice
    Invoice invoice = new Invoice();
    invoice.setId("inv_" + UUID.randomUUID().toString().substring(0, 8));
    invoice.setSubscription(subscription);
    invoice.setStatus(InvoiceStatus.DRAFT);
    invoice.setDueDate(OffsetDateTime.now().plusDays(7));
    invoice.setAmount(BigDecimal.ZERO); // Will update after line items
    invoiceRepository.save(invoice);

    // Line Item 1: Flat fee (for FIXED or TIERED plans)
    if (plan.getBillingType() == BillingType.FIXED || plan.getBillingType() == BillingType.TIERED) {
        InvoiceLineItem flatItem = new InvoiceLineItem();
        flatItem.setId("ili_" + UUID.randomUUID().toString().substring(0, 8));
        flatItem.setInvoice(invoice);
        flatItem.setPlan(plan);
        flatItem.setType(InvoiceLineItemType.FLAT);
        flatItem.setAmount(plan.getPrice());
        flatItem.setDescription("Monthly subscription: " + plan.getName());
        invoiceLineItemRepository.save(flatItem);
        totalAmount = totalAmount.add(plan.getPrice());
    }

    // Line Item 2: Usage aggregation (for USAGE_BASED plans)
    if (plan.getBillingType() == BillingType.USAGE_BASED) {
        BigDecimal totalUsage = usageEventRepository.sumUsageForPeriod(
            subscription.getId(), periodStart, periodEnd);

        BigDecimal usageCost = totalUsage.multiply(plan.getPrice()).setScale(4, RoundingMode.HALF_UP);

        InvoiceLineItem usageItem = new InvoiceLineItem();
        usageItem.setId("ili_" + UUID.randomUUID().toString().substring(0, 8));
        usageItem.setInvoice(invoice);
        usageItem.setPlan(plan);
        usageItem.setType(InvoiceLineItemType.USAGE);
        usageItem.setAmount(usageCost);
        usageItem.setDescription(totalUsage.toPlainString() + " units × $" + plan.getPrice().toPlainString() + "/unit");
        invoiceLineItemRepository.save(usageItem);
        totalAmount = totalAmount.add(usageCost);
    }

    // Update invoice total
    invoice.setAmount(totalAmount);
    invoiceRepository.save(invoice);

    if (totalAmount.compareTo(BigDecimal.ZERO) <= 0) {
        // No charge needed – mark paid and extend period
        invoice.setStatus(InvoiceStatus.PAID);
        invoiceRepository.save(invoice);
        extendSubscriptionPeriod(subscription);
        return;
    }

    // Attempt payment
    String cycleIdempotencyKey = "cycle_" + subscription.getId() + "_" + periodEnd.toInstant().toEpochMilli();
    PaymentTransaction tx = paymentService.processPayment(subscription, invoice, "pm_default", cycleIdempotencyKey);

    if (tx.getStatus() == PaymentStatus.SUCCESS) {
        extendSubscriptionPeriod(subscription);
        log.info("Cycle billing succeeded for subscription: {}", subscription.getId());
    } else {
        subscription.setStatus(SubscriptionStatus.PAST_DUE);
        subscriptionRepository.save(subscription);

        writeOutboxEvent("subscription.changed", Map.of(

```

```
        "subscriptionId", subscription.getId(),
        "planId", subscription.getPlan().getId(),
        "status", SubscriptionStatus.PAST_DUE.name(),
        "reason", "Cycle billing payment failed"
    ));
    log.warn("Cycle billing failed for subscription: {}. Status set to PAST_DUE.", subscription.getId());
}

private void extendSubscriptionPeriod(Subscription subscription) {
    OffsetDateTime oldEnd = subscription.getCurrentPeriodEnd();
    OffsetDateTime newStart = oldEnd;
    OffsetDateTime newEnd;

    switch (subscription.getPlan().getBillingPeriod()) {
        case MONTHLY -> newEnd = oldEnd.plusMonths(1);
        case YEARLY -> newEnd = oldEnd.plusYears(1);
        default -> newEnd = oldEnd.plusMonths(1);
    }

    subscription.setCurrentPeriodStart(newStart);
    subscription.setCurrentPeriodEnd(newEnd);
    subscription.setStatus(SubscriptionStatus.ACTIVE);
    subscriptionRepository.save(subscription);
    log.info("Extended subscription {} period: {} to {}", subscription.getId(), newStart, newEnd);
}

private void writeOutboxEvent(String eventType, Object payload) {
    try {
        String jsonPayload = objectMapper.writeValueAsString(payload);
        OutboxEvent event = new OutboxEvent();
        event.setId("evt_" + UUID.randomUUID().toString().substring(0, 8));
        event.setEventType(eventType);
        event.setPayload(jsonPayload);
        event.setStatus(OutboxEventStatus.PENDING);
        outboxEventRepository.save(event);
    } catch (Exception e) {
        log.error("Failed to write outbox event", e);
    }
}
```

File: src/main/java/com/subsflow/billing/scheduler/BillingCycleScheduler.java

```
package com.subsflow.billing.scheduler;

import com.subsflow.billing.service.BillingCycleService;
import com.subsflow.common.context.TenantContext;
import com.subsflow.tenant.entity.Tenant;
import com.subsflow.tenant.repository.TenantRepository;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.scheduling.annotation.EnableScheduling;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Component;

import java.util.List;

@Component
@EnableScheduling
public class BillingCycleScheduler {

    private static final Logger log = LoggerFactory.getLogger(BillingCycleScheduler.class);

    private final TenantRepository tenantRepository;
    private final BillingCycleService billingCycleService;

    public BillingCycleScheduler(TenantRepository tenantRepository,
                               BillingCycleService billingCycleService) {
        this.tenantRepository = tenantRepository;
        this.billingCycleService = billingCycleService;
    }

    @Scheduled(fixedDelay = 10000)
    public void runBillingCycle() {
        List<Tenant> tenants = tenantRepository.findAll();
        for (Tenant tenant : tenants) {
            try {
                TenantContext.setTenantId(tenant.getId());
                billingCycleService.billDueSubscriptions(tenant.getId());
            } catch (Exception e) {
                log.error("Error running billing cycle for tenant: {}", tenant.getId(), e);
            } finally {
                TenantContext.clear();
            }
        }
    }
}
```

File: src/main/java/com/subsflow/billing/controller/InvoiceController.java

```
package com.subsflow.billing.controller;

import com.subsflow.billing.entity.Invoice;
import com.subsflow.billing.entity.InvoiceLineItem;
import com.subsflow.billing.repository.InvoiceRepository;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.math.BigDecimal;
import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;

@RestController
@RequestMapping("/api/v1/invoices")
public class InvoiceController {

    private final InvoiceRepository invoiceRepository;

    public InvoiceController(InvoiceRepository invoiceRepository) {
        this.invoiceRepository = invoiceRepository;
    }

    @GetMapping
    public ResponseEntity<List<InvoiceSummaryResponse>> listInvoices() {
        List<InvoiceSummaryResponse> invoices = invoiceRepository.findAll().stream()
            .map(InvoiceSummaryResponse::from)
            .collect(Collectors.toList());
        return ResponseEntity.ok(invoices);
    }

    @GetMapping("/{id}")
    public ResponseEntity<?> getInvoice(@PathVariable("id") String id) {
        return invoiceRepository.findById(id)
            .map(inv -> ResponseEntity.ok(InvoiceDetailResponse.from(inv)))
            .orElseGet(() -> ResponseEntity.notFound().build());
    }

    public static class InvoiceSummaryResponse {
        protected String id;
        protected String subscriptionId;
        protected String status;
        protected BigDecimal amount;
        protected String dueDate;
        protected String createdAt;

        public static InvoiceSummaryResponse from(Invoice invoice) {
            InvoiceSummaryResponse dto = new InvoiceSummaryResponse();
            dto.id = invoice.getId();
            dto.subscriptionId = invoice.getSubscription() != null ? invoice.getSubscription().getId() : null;
            dto.status = invoice.getStatus().name();
            dto.amount = invoice.getAmount();
            dto.dueDate = invoice.getDueDate() != null ? invoice.getDueDate().toString() : null;
            dto.createdAt = invoice.getCreatedAt() != null ? invoice.getCreatedAt().toString() : null;
            return dto;
        }

        public String getId() { return id; }
        public String getSubscriptionId() { return subscriptionId; }
        public String getStatus() { return status; }
        public BigDecimal getAmount() { return amount; }
        public String getDueDate() { return dueDate; }
        public String getCreatedAt() { return createdAt; }
    }

    public static class InvoiceDetailResponse extends InvoiceSummaryResponse {
        private List<LineItemResponse> lineItems;

        public static InvoiceDetailResponse from(Invoice invoice) {
            InvoiceDetailResponse dto = new InvoiceDetailResponse();
            dto.id = invoice.getId();
            dto.subscriptionId = invoice.getSubscription() != null ? invoice.getSubscription().getId() : null;
            dto.status = invoice.getStatus().name();
            dto.amount = invoice.getAmount();
            dto.dueDate = invoice.getDueDate() != null ? invoice.getDueDate().toString() : null;
            dto.createdAt = invoice.getCreatedAt() != null ? invoice.getCreatedAt().toString() : null;
            dto.lineItems = invoice.getLineItems().stream()

```

```
        .map(LineItemResponse::from)
        .collect(Collectors.toList());
    return dto;
}

    public List<LineItemResponse> getLineItems() { return lineItems; }
}

public static class LineItemResponse {
    private String id;
    private String type;
    private BigDecimal amount;
    private String description;
    private String planId;

    public static LineItemResponse from(InvoiceLineItem item) {
        LineItemResponse dto = new LineItemResponse();
        dto.id = item.getId();
        dto.type = item.getType().name();
        dto.amount = item.getAmount();
        dto.description = item.getDescription();
        dto.planId = item.getPlan() != null ? item.getPlan().getId() : null;
        return dto;
    }

    public String getId() { return id; }
    public String getType() { return type; }
    public BigDecimal getAmount() { return amount; }
    public String getDescription() { return description; }
    public String getPlanId() { return planId; }
}
}
```

File: src/main/java/com/subsflow/payment/entity/IdempotencyKeyId.java

```
package com.subsflow.payment.entity;

import java.io.Serializable;
import java.util.Objects;

public class IdempotencyKeyId implements Serializable {
    private String tenantId;
    private String key;

    public IdempotencyKeyId() {}

    public IdempotencyKeyId(String tenantId, String key) {
        this.tenantId = tenantId;
        this.key = key;
    }

    public String getTenantId() {
        return tenantId;
    }

    public void setTenantId(String tenantId) {
        this.tenantId = tenantId;
    }

    public String getKey() {
        return key;
    }

    public void setKey(String key) {
        this.key = key;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        IdempotencyKeyId that = (IdempotencyKeyId) o;
        return Objects.equals(tenantId, that.tenantId) && Objects.equals(key, that.key);
    }

    @Override
    public int hashCode() {
        return Objects.hash(tenantId, key);
    }
}
```

File: src/main/java/com/subsflow/payment/entity/IdempotencyKeyStatus.java

```
package com.subsflow.payment.entity;

public enum IdempotencyKeyStatus {
    IN_PROGRESS,
    COMPLETED
}
```


File: `src/main/java/com/subsflow/payment/entity/IdempotencyKey.java`

```
package com.subsflow.payment.entity;

import jakarta.persistence.*;
import lombok.Getter;
import lombok.Setter;
import org.hibernate.annotations.TenantId;

import java.time.OffsetDateTime;

@Entity
@Table(name = "idempotency_key")
@IdClass(IdempotencyKeyId.class)
@Getter
@Setter
public class IdempotencyKey {

    @Id
    @Column(name = "key", nullable = false)
    private String key;

    @Id
    @TenantId
    @Column(name = "tenant_id", nullable = false, updatable = false)
    private String tenantId;

    @Column(name = "request_hash", nullable = false, length = 64)
    private String requestHash;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private IdempotencyKeyStatus status;

    @Column(name = "response_payload", columnDefinition = "TEXT")
    private String responsePayload;

    @Column(name = "created_at", nullable = false, updatable = false)
    private OffsetDateTime createdAt = OffsetDateTime.now();
}
```

File: `src/main/java/com/subsflow/payment/entity/PaymentStatus.java`

```
package com.subsflow.payment.entity;

public enum PaymentStatus {
    PENDING,
    SUCCESS,
    FAILED
}
```

File: src/main/java/com/subsflow/payment/entity/PaymentTransaction.java

```
package com.subsflow.payment.entity;

import com.subsflow.common.entity.AbstractTenantEntity;
import com.subsflow.billing.entity.Invoice;
import com.subsflow.subscription.entity.Subscription;
import jakarta.persistence.*;
import lombok.Getter;
import lombok.Setter;

import java.math.BigDecimal;
import java.time.OffsetDateTime;

@Entity
@Table(name = "payment_transaction")
@Getter
@Setter
public class PaymentTransaction extends AbstractTenantEntity {

    @Id
    private String id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "subscription_id")
    private Subscription subscription;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "invoice_id")
    private Invoice invoice;

    @Column(nullable = false, precision = 20, scale = 4)
    private BigDecimal amount;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private PaymentStatus status;

    @Column(name = "idempotency_key")
    private String idempotencyKey;

    @Column(name = "gateway_reference")
    private String gatewayReference;

    @Column(name = "error_message", columnDefinition = "TEXT")
    private String errorMessage;

    @Column(name = "created_at", nullable = false, updatable = false)
    private OffsetDateTime createdAt = OffsetDateTime.now();

    @Column(name = "updated_at", nullable = false)
    private OffsetDateTime updatedAt = OffsetDateTime.now();

    @PreUpdate
    protected void onUpdate() {
        updatedAt = OffsetDateTime.now();
    }
}
```

File: `src/main/java/com/subsflow/payment/gateway/ChargeRequest.java`

```
package com.subsflow.payment.gateway;

import java.math.BigDecimal;

public class ChargeRequest {
    private final BigDecimal amount;
    private final String currency;
    private final String paymentMethodId;

    public ChargeRequest(BigDecimal amount, String currency, String paymentMethodId) {
        this.amount = amount;
        this.currency = currency;
        this.paymentMethodId = paymentMethodId;
    }

    public BigDecimal getAmount() {
        return amount;
    }

    public String getCurrency() {
        return currency;
    }

    public String getPaymentMethodId() {
        return paymentMethodId;
    }
}
```

File: `src/main/java/com/subsflow/payment/gateway/ChargeResponse.java`

```
package com.subsflow.payment.gateway;

public class ChargeResponse {
    private final boolean success;
    private final String gatewayReference;
    private final String errorMessage;

    public ChargeResponse(boolean success, String gatewayReference, String errorMessage) {
        this.success = success;
        this.gatewayReference = gatewayReference;
        this.errorMessage = errorMessage;
    }

    public boolean isSuccess() {
        return success;
    }

    public String getGatewayReference() {
        return gatewayReference;
    }

    public String getErrorMessage() {
        return errorMessage;
    }
}
```

File: `src/main/java/com/subsflow/payment/gateway/PaymentGatewayClient.java`

```
package com.subsflow.payment.gateway;

public interface PaymentGatewayClient {
    /**
     * Executes a charge request against the payment gateway.
     * @throws Exception representing transient connection failures or timeouts
     */
    ChargeResponse charge(ChargeRequest request) throws Exception;
}
```

File: `src/main/java/com/subsflow/payment/gateway/SandboxPaymentGatewayClient.java`

```
package com.subsflow.payment.gateway;

import org.springframework.stereotype.Component;

import java.util.UUID;
import java.util.concurrent.TimeoutException;

@Component
public class SandboxPaymentGatewayClient implements PaymentGatewayClient {

    @Override
    public ChargeResponse charge(ChargeRequest request) throws Exception {
        String paymentMethod = request.getPaymentMethodId();

        if (paymentMethod != null) {
            if (paymentMethod.endsWith("0000")) {
                // Card decline / insufficient funds failure
                return new ChargeResponse(false, null, "Insufficient funds");
            }
            if (paymentMethod.endsWith("9999")) {
                // Simulated gateway latency / timeout
                Thread.sleep(3000);
                throw new TimeoutException("Simulated connection timeout from Sandbox Payment Gateway");
            }
        }

        // Default: Success
        return new ChargeResponse(true, "gw_tx_" + UUID.randomUUID().toString().substring(0, 8), null);
    }
}
```

File: `src/main/java/com/subsflow/payment/repository/IdempotencyKeyRepository.java`

```
package com.subsflow.payment.repository;

import com.subsflow.payment.entity.IdempotencyKey;
import com.subsflow.payment.entity.IdempotencyKeyId;
import org.springframework.data.jpa.repository.JpaRepository;

public interface IdempotencyKeyRepository extends JpaRepository<IdempotencyKey, IdempotencyKeyId> {
}
```

File: `src/main/java/com/subsflow/payment/repository/PaymentTransactionRepository.java`

```
package com.subsflow.payment.repository;

import com.subsflow.payment.entity.PaymentTransaction;
import org.springframework.data.jpa.repository.JpaRepository;

public interface PaymentTransactionRepository extends JpaRepository<PaymentTransaction, String> {
}
```

File: `src/main/java/com/subsflow/payment/service/IdempotencyService.java`

```
package com.subsflow.payment.service;

import com.subsflow.common.exception.IdempotencyConflictException;
import com.subsflow.common.exception.IdempotencyKeyUsedException;

public interface IdempotencyService {

    /**
     * Attempts to start an idempotent operation.
     * Inserts an idempotency key with status IN_PROGRESS.
     *
     * @param key the idempotency key
     * @param requestHash the SHA-256 hash of the request payload
     * @throws IdempotencyConflictException if the key is already processing (status IN_PROGRESS).
     * @throws IdempotencyKeyUsedException if the key has completed (status COMPLETED, wraps cached response).
     */
    void startOperation(String key, String requestHash);

    /**
     * Completes the operation, saving the response payload.
     *
     * @param key the idempotency key
     * @param responsePayload the response payload to cache
     */
    void completeOperation(String key, String responsePayload);
}
```

File: src/main/java/com/subsflow/payment/service/IdempotencyServiceImpl.java

```
package com.subsflow.payment.service;

import com.subsflow.common.context.TenantContext;
import com.subsflow.common.exception.IdempotencyConflictException;
import com.subsflow.common.exception.IdempotencyKeyUsedException;
import com.subsflow.payment.entity.IdempotencyKey;
import com.subsflow.payment.entity.IdempotencyKeyId;
import com.subsflow.payment.entity.IdempotencyKeyStatus;
import com.subsflow.payment.repository.IdempotencyKeyRepository;
import org.springframework.dao.DataIntegrityViolationException;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Propagation;
import org.springframework.transaction.annotation.Transactional;

import java.util.Optional;

@Service
public class IdempotencyServiceImpl implements IdempotencyService {

    private final IdempotencyKeyRepository idempotencyKeyRepository;

    public IdempotencyServiceImpl(IdempotencyKeyRepository idempotencyKeyRepository) {
        this.idempotencyKeyRepository = idempotencyKeyRepository;
    }

    @Override
    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public void startOperation(String key, String requestHash) {
        String tenantId = TenantContext.getTenantId();
        if (tenantId == null) {
            throw new IllegalStateException("Tenant context not found");
        }

        IdempotencyKey idempotency = new IdempotencyKey();
        idempotency.setKey(key);
        idempotency.setTenantId(tenantId);
        idempotency.setRequestHash(requestHash);
        idempotency.setStatus(IdempotencyKeyStatus.IN_PROGRESS);

        try {
            // Attempt to insert. If key exists, it will throw DataIntegrityViolationException.
            idempotencyKeyRepository.saveAndFlush(idempotency);
        } catch (DataIntegrityViolationException e) {
            // Unique constraint violation occurred. Look up existing record to determine status.
            IdempotencyKeyId id = new IdempotencyKeyId(tenantId, key);
            Optional<IdempotencyKey> existingOpt = idempotencyKeyRepository.findById(id);
            if (existingOpt.isPresent()) {
                IdempotencyKey existing = existingOpt.get();

                // If it exists but request hash differs, it's key misuse.
                if (!existing.getRequestHash().equals(requestHash)) {
                    throw new IdempotencyConflictException("Idempotency key was used with a different request payload");
                }

                if (existing.getStatus() == IdempotencyKeyStatus.IN_PROGRESS) {
                    throw new IdempotencyConflictException("Operation is already in progress");
                } else if (existing.getStatus() == IdempotencyKeyStatus.COMPLETED) {
                    throw new IdempotencyKeyUsedException("Operation already completed", existing.getResponsePayload());
                }
            }
            throw new IdempotencyConflictException("Duplicate request");
        }
    }

    @Override
    @Transactional
    public void completeOperation(String key, String responsePayload) {
        String tenantId = TenantContext.getTenantId();
        if (tenantId == null) {
            throw new IllegalStateException("Tenant context not found");
        }

        IdempotencyKeyId id = new IdempotencyKeyId(tenantId, key);
        IdempotencyKey idempotency = idempotencyKeyRepository.findById(id)
            .orElseThrow(() -> new IllegalStateException("Idempotency record not found for completion"));

        idempotency.setStatus(IdempotencyKeyStatus.COMPLETED);
    }
}
```

```

        idempotency.setResponsePayload(responsePayload);
        idempotencyKeyRepository.save(idempotency);
    }
}

```

File: `src/main/java/com/subsflow/payment/service/PaymentGatewayResilienceService.java`

```

package com.subsflow.payment.service;

import com.subsflow.payment.gateway.ChargeRequest;
import com.subsflow.payment.gateway.ChargeResponse;
import com.subsflow.payment.gateway.PaymentGatewayClient;
import io.github.resilience4j.circuitbreaker.annotation.CircuitBreaker;
import io.github.resilience4j.retry.annotation.Retry;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Service;

@Service
public class PaymentGatewayResilienceService {

    private static final Logger log = LoggerFactory.getLogger(PaymentGatewayResilienceService.class);

    private final PaymentGatewayClient gatewayClient;

    public PaymentGatewayResilienceService(PaymentGatewayClient gatewayClient) {
        this.gatewayClient = gatewayClient;
    }

    @CircuitBreaker(name = "paymentGateway", fallbackMethod = "fallbackCharge")
    @Retry(name = "paymentGateway")
    public ChargeResponse charge(ChargeRequest request) throws Exception {
        return gatewayClient.charge(request);
    }

    public ChargeResponse fallbackCharge(ChargeRequest request, Throwable throwable) {
        log.warn("Payment gateway fallback triggered for request {} due to {}", request.getPaymentMethodId(),
            throwable.getMessage());
        return new ChargeResponse(false, null, "Payment Gateway Unavailable");
    }
}

```

File: `src/main/java/com/subsflow/payment/service/PaymentService.java`

```

package com.subsflow.payment.service;

import com.subsflow.billing.entity.Invoice;
import com.subsflow.payment.entity.PaymentTransaction;
import com.subsflow.subscription.entity.Subscription;

public interface PaymentService {
    /**
     * Attempts to charge a customer's payment method for a specific invoice.
     */
    PaymentTransaction processPayment(Subscription subscription, Invoice invoice, String paymentMethodId, String
        idempotencyKey);
}

```

File: src/main/java/com/subsflow/payment/service/PaymentServiceImpl.java

```
package com.subsflow.payment.service;

import com.fasterxml.jackson.databind.ObjectMapper;
import com.subsflow.billing.entity.Invoice;
import com.subsflow.billing.entity.InvoiceStatus;
import com.subsflow.billing.repository.InvoiceRepository;
import com.subsflow.common.entity.OutboxEvent;
import com.subsflow.common.entity.OutboxEventStatus;
import com.subsflow.common.repository.OutboxEventRepository;
import com.subsflow.dunning.entity.PaymentRetryQueue;
import com.subsflow.dunning.entity.RetryStatus;
import com.subsflow.dunning.repository.PaymentRetryQueueRepository;
import com.subsflow.payment.entity.PaymentStatus;
import com.subsflow.payment.entity.PaymentTransaction;
import com.subsflow.payment.gateway.ChargeRequest;
import com.subsflow.payment.gateway.ChargeResponse;
import com.subsflow.payment.gateway.PaymentGatewayClient;
import com.subsflow.payment.repository.PaymentTransactionRepository;
import com.subsflow.subscription.entity.Subscription;
import io.github.resilience4j.circuitbreaker.annotation.CircuitBreaker;
import io.github.resilience4j.retry.annotation.Retry;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.time.OffsetDateTime;
import java.util.Map;
import java.util.UUID;

@Service
public class PaymentServiceImpl implements PaymentService {

    private static final Logger log = LoggerFactory.getLogger(PaymentServiceImpl.class);

    private final PaymentGatewayClient gatewayClient;
    private final PaymentTransactionRepository transactionRepository;
    private final PaymentRetryQueueRepository retryQueueRepository;
    private final OutboxEventRepository outboxEventRepository;
    private final InvoiceRepository invoiceRepository;
    private final ObjectMapper objectMapper;
    private final com.subsflow.common.metrics.MetricsService metricsService;

    public PaymentServiceImpl(PaymentGatewayClient gatewayClient,
                             PaymentTransactionRepository transactionRepository,
                             PaymentRetryQueueRepository retryQueueRepository,
                             OutboxEventRepository outboxEventRepository,
                             InvoiceRepository invoiceRepository,
                             ObjectMapper objectMapper,
                             com.subsflow.common.metrics.MetricsService metricsService) {
        this.gatewayClient = gatewayClient;
        this.transactionRepository = transactionRepository;
        this.retryQueueRepository = retryQueueRepository;
        this.outboxEventRepository = outboxEventRepository;
        this.invoiceRepository = invoiceRepository;
        this.objectMapper = objectMapper;
        this.metricsService = metricsService;
    }

    @Override
    @Transactional
    @CircuitBreaker(name = "paymentGateway", fallbackMethod = "handlePaymentFallback")
    @Retry(name = "paymentGateway")
    public PaymentTransaction processPayment(Subscription subscription, Invoice invoice, String paymentMethodId, String
    idempotencyKey) {
        log.info("Initiating payment gateway charge for subscription: {} and invoice: {}", subscription.getId(),
        invoice.getId());

        ChargeRequest request = new ChargeRequest(invoice.getAmount(), "USD", paymentMethodId);

        ChargeResponse response;
        try {
            response = gatewayClient.charge(request);
        } catch (Exception e) {
            log.warn("Payment gateway call failed for invoice {}", invoice.getId(), e);
            throw new RuntimeException(e);
        }
    }
}
```



```

PaymentTransaction transaction = new PaymentTransaction();
transaction.setId("tx_" + UUID.randomUUID().toString().substring(0, 8));
transaction.setSubscription(subscription);
transaction.setInvoice(invoice);
transaction.setAmount(invoice.getAmount());
transaction.setIdempotencyKey(idempotencyKey);

if (response.isSuccess()) {
    transaction.setStatus(PaymentStatus.SUCCESS);
    transaction.setGatewayReference(response.getGatewayReference());
    transactionRepository.save(transaction);

    // Update invoice status to PAID
    invoice.setStatus(InvoiceStatus.PAID);
    invoiceRepository.save(invoice);

    // Write outbox event
    writeOutboxEvent("payment.succeeded", Map.of(
        "transactionId", transaction.getId(),
        "subscriptionId", subscription.getId(),
        "invoiceId", invoice.getId(),
        "amount", invoice.getAmount(),
        "gatewayReference", response.getGatewayReference()
    ));

    metricsService.incrementPaymentSuccess();
    log.info("Payment succeeded! Tx: {}", transaction.getId());
} else {
    transaction.setStatus(PaymentStatus.FAILED);
    transaction.setErrorMessage(response.getErrorMessage());
    transactionRepository.save(transaction);

    // Update invoice status to UNPAID
    invoice.setStatus(InvoiceStatus.UNPAID);
    invoiceRepository.save(invoice);

    // Enqueue for dunning
    enqueueDunning(transaction);

    // Write outbox event
    writeOutboxEvent("payment.failed", Map.of(
        "transactionId", transaction.getId(),
        "subscriptionId", subscription.getId(),
        "invoiceId", invoice.getId(),
        "amount", invoice.getAmount(),
        "errorMessage", response.getErrorMessage()
    ));

    metricsService.incrementPaymentFailed();
    log.warn("Payment declined: {}", response.getErrorMessage());
}

return transaction;
}

// Fallback method when payment fails (e.g. gateway down, timeouts, circuit open)
@Transactional
public PaymentTransaction handlePaymentFallback(Subscription subscription, Invoice invoice, String paymentMethodId, String
idempotencyKey, Throwable t) {
    log.error("Payment fallback triggered due to exception: {}", t.getMessage());

    PaymentTransaction transaction = new PaymentTransaction();
    transaction.setId("tx_" + UUID.randomUUID().toString().substring(0, 8));
    transaction.setSubscription(subscription);
    transaction.setInvoice(invoice);
    transaction.setAmount(invoice.getAmount());
    transaction.setIdempotencyKey(idempotencyKey);
    transaction.setStatus(PaymentStatus.FAILED);
    transaction.setErrorMessage("Payment Gateway Timeout / Unhealthy: " + t.getMessage());
    transactionRepository.save(transaction);

    // Update invoice to UNPAID
    invoice.setStatus(InvoiceStatus.UNPAID);
    invoiceRepository.save(invoice);

    // Enqueue for dunning retry
    enqueueDunning(transaction);
}

```

```

        // Write outbox event
        writeOutboxEvent("payment.failed", Map.of(
            "transactionId", transaction.getId(),
            "subscriptionId", subscription.getId(),
            "invoiceId", invoice.getId(),
            "amount", invoice.getAmount(),
            "errorMessage", transaction.getErrorMessage()
        ));

        metricsService.incrementPaymentFailed();
        return transaction;
    }

    private void enqueueDunning(PaymentTransaction transaction) {
        PaymentRetryQueue retryQueue = new PaymentRetryQueue();
        retryQueue.setId("dunning_" + UUID.randomUUID().toString().substring(0, 8));
        retryQueue.setTransaction(transaction);
        retryQueue.setRetryCount(0);
        // First retry in 1 minute for test simulation
        retryQueue.setNextRetryAt(OffsetDateTime.now().plusMinutes(1));
        retryQueue.setStatus(RetryStatus.PENDING);
        retryQueueRepository.save(retryQueue);
        log.info("Transaction enqueued for dunning retry: {}", retryQueue.getId());
    }

    private void writeOutboxEvent(String eventType, Object payload) {
        try {
            String jsonPayload = objectMapper.writeValueAsString(payload);
            OutboxEvent event = new OutboxEvent();
            event.setId("evt_" + UUID.randomUUID().toString().substring(0, 8));
            event.setEventType(eventType);
            event.setPayload(jsonPayload);
            event.setStatus(OutboxEventStatus.PENDING);
            outboxEventRepository.save(event);
        } catch (Exception e) {
            log.error("Failed to write outbox event", e);
        }
    }
}

```

File: `src/main/java/com/leu/subsflow/dunning/entity/RetryStatus.java`

```

package com.subsflow.dunning.entity;

public enum RetryStatus {
    PENDING,
    PROCESSING,
    COMPLETED,
    FAILED
}

```

File: `src/main/java/com/subsflow/dunning/entity/PaymentRetryQueue.java`

```
package com.subsflow.dunning.entity;

import com.subsflow.common.entity.AbstractTenantEntity;
import com.subsflow.payment.entity.PaymentTransaction;
import jakarta.persistence.*;
import lombok.Getter;
import lombok.Setter;

import java.time.OffsetDateTime;

@Entity
@Table(name = "payment_retry_queue")
@Getter
@Setter
public class PaymentRetryQueue extends AbstractTenantEntity {

    @Id
    private String id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "transaction_id", nullable = false)
    private PaymentTransaction transaction;

    @Column(name = "retry_count", nullable = false)
    private int retryCount = 0;

    @Column(name = "next_retry_at", nullable = false)
    private OffsetDateTime nextRetryAt;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private RetryStatus status;

    @Column(name = "created_at", nullable = false, updatable = false)
    private OffsetDateTime createdAt = OffsetDateTime.now();
}
```

File: `src/main/java/com/subsflow/dunning/repository/PaymentRetryQueueRepository.java`

```
package com.subsflow.dunning.repository;

import com.subsflow.dunning.entity.PaymentRetryQueue;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import java.time.OffsetDateTime;
import java.util.List;

public interface PaymentRetryQueueRepository extends JpaRepository<PaymentRetryQueue, String> {

    @Query(value = "SELECT * FROM payment_retry_queue WHERE status = 'PENDING' AND next_retry_at <= :now ORDER BY created_at ASC LIMIT 50 FOR UPDATE SKIP LOCKED", nativeQuery = true)
    List<PaymentRetryQueue> findPendingRetries(OffsetDateTime now);
}
```

File: src/main/java/com/subsflow/dunning/service/DunningScheduler.java

```
package com.subsflow.dunning.service;

import com.subsflow.billing.entity.Invoice;
import com.subsflow.billing.entity.InvoiceStatus;
import com.subsflow.billing.repository.InvoiceRepository;
import com.subsflow.common.context.TenantContext;
import com.subsflow.common.entity.OutboxEvent;
import com.subsflow.common.entity.OutboxEventStatus;
import com.subsflow.common.repository.OutboxEventRepository;
import com.subsflow.dunning.entity.PaymentRetryQueue;
import com.subsflow.dunning.entity.RetryStatus;
import com.subsflow.dunning.repository.PaymentRetryQueueRepository;
import com.subsflow.payment.entity.PaymentStatus;
import com.subsflow.payment.entity.PaymentTransaction;
import com.subsflow.payment.service.PaymentService;
import com.subsflow.subscription.entity.Subscription;
import com.subsflow.subscription.entity.SubscriptionStatus;
import com.subsflow.subscription.repository.SubscriptionRepository;
import com.subsflow.tenant.entity.Tenant;
import com.subsflow.tenant.repository.TenantRepository;
import com.fasterxml.jackson.databind.ObjectMapper;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.scheduling.annotation.EnableScheduling;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Component;
import org.springframework.transaction.annotation.Propagation;
import org.springframework.transaction.annotation.Transactional;

import java.time.OffsetDateTime;
import java.util.List;
import java.util.Map;
import java.util.UUID;

@Component
@EnableScheduling
public class DunningScheduler {

    private static final Logger log = LoggerFactory.getLogger(DunningScheduler.class);
    private static final int MAX_RETRIES = 3;
    private static final int[] BACKOFF_MINUTES = {1, 3, 7};

    private final TenantRepository tenantRepository;
    private final PaymentRetryQueueRepository retryQueueRepository;
    private final PaymentService paymentService;
    private final SubscriptionRepository subscriptionRepository;
    private final InvoiceRepository invoiceRepository;
    private final OutboxEventRepository outboxEventRepository;
    private final ObjectMapper objectMapper;
    private final com.subsflow.common.metrics.MetricsService metricsService;

    public DunningScheduler(TenantRepository tenantRepository,
                           PaymentRetryQueueRepository retryQueueRepository,
                           PaymentService paymentService,
                           SubscriptionRepository subscriptionRepository,
                           InvoiceRepository invoiceRepository,
                           OutboxEventRepository outboxEventRepository,
                           ObjectMapper objectMapper,
                           com.subsflow.common.metrics.MetricsService metricsService) {
        this.tenantRepository = tenantRepository;
        this.retryQueueRepository = retryQueueRepository;
        this.paymentService = paymentService;
        this.subscriptionRepository = subscriptionRepository;
        this.invoiceRepository = invoiceRepository;
        this.outboxEventRepository = outboxEventRepository;
        this.objectMapper = objectMapper;
        this.metricsService = metricsService;
    }

    @Scheduled(fixedDelay = 5000)
    public void processDunningQueue() {
        List<Tenant> tenants = tenantRepository.findAll();
        for (Tenant tenant : tenants) {
            try {
                processTenantRetries(tenant.getId());
            } catch (Exception e) {
                log.error("Error processing dunning retries for tenant: {}", tenant.getId(), e);
            }
        }
    }
}
```

```

    }
}

@Transactional(propagation = Propagation.REQUIRES_NEW)
public void processTenantRetries(String tenantId) {
    try {
        TenantContext.setTenantId(tenantId);
        OffsetDateTime now = OffsetDateTime.now();
        List<PaymentRetryQueue> pendingRetries = retryQueueRepository.findPendingRetries(now);

        if (pendingRetries.isEmpty()) {
            return;
        }

        log.info("Processing {} pending dunning retries for tenant: {}", pendingRetries.size(), tenantId);

        for (PaymentRetryQueue retry : pendingRetries) {
            PaymentTransaction oldTx = retry.getTransaction();
            Subscription subscription = oldTx.getSubscription();
            Invoice invoice = oldTx.getInvoice();

            // If invoice is already paid, complete this queue item
            if (invoice.getStatus() == InvoiceStatus.PAID) {
                retry.setStatus(RetryStatus.COMPLETED);
                retryQueueRepository.save(retry);
                continue;
            }

            log.info("Dunning execution: Attempt {} for invoice {}", retry.getRetryCount() + 1, invoice.getId());

            // Generate new unique idempotency key for this retry attempt
            String retryIdempotencyKey = "retry_" + invoice.getId() + "_" + retry.getRetryCount();

            // Process the retry payment. We default to standard mock payment method or retry original.
            String pmId = oldTx.getIdempotencyKey() != null ? "pm_retry_card" : "pm_default";
            metricsService.incrementDunningRetry();
            PaymentTransaction newTx = paymentService.processPayment(subscription, invoice, pmId, retryIdempotencyKey);

            if (newTx.getStatus() == PaymentStatus.SUCCESS) {
                retry.setStatus(RetryStatus.COMPLETED);
                retryQueueRepository.save(retry);

                subscription.setStatus(SubscriptionStatus.ACTIVE);
                subscriptionRepository.save(subscription);
                log.info("Dunning succeeded! Invoice {} paid on attempt {}", invoice.getId(), retry.getRetryCount() + 1);
            } else {
                int attempts = retry.getRetryCount() + 1;
                retry.setRetryCount(attempts);

                if (attempts >= MAX_RETRIES) {
                    retry.setStatus(RetryStatus.FAILED);
                    retryQueueRepository.save(retry);

                    // Suspend Subscription
                    subscription.setStatus(SubscriptionStatus.SUSPENDED);
                    subscriptionRepository.save(subscription);

                    // Write Outbox Event for suspension
                    writeOutboxEvent("subscription.changed", Map.of(
                        "subscriptionId", subscription.getId(),
                        "oldPlanId", subscription.getPlan().getId(),
                        "newPlanId", subscription.getPlan().getId(),
                        "status", SubscriptionStatus.SUSPENDED.name(),
                        "reason", "Dunning max retries exceeded"
                    ));
                    log.warn("Dunning failed! Max retries exceeded. Subscription {} SUSPENDED", subscription.getId());
                } else {
                    int backoffMinutes = BACKOFF_MINUTES[Math.min(attempts - 1, BACKOFF_MINUTES.length - 1)];
                    retry.setNextRetryAt(OffsetDateTime.now().plusMinutes(backoffMinutes));
                    retry.setStatus(RetryStatus.PENDING);
                    retryQueueRepository.save(retry);
                    log.info("Dunning retry failed. Rescheduling attempt {} for {}", attempts + 1,
                        retry.getId());
                }
            }
        }
    } finally {

```

```

        TenantContext.clear();
    }
}

private void writeOutboxEvent(String eventType, Object payload) {
    try {
        String jsonPayload = objectMapper.writeValueAsString(payload);
        OutboxEvent event = new OutboxEvent();
        event.setId("evt_" + UUID.randomUUID().toString().substring(0, 8));
        event.setEventType(eventType);
        event.setPayload(jsonPayload);
        event.setStatus(OutboxEventStatus.PENDING);
        outboxEventRepository.save(event);
    } catch (Exception e) {
        log.error("Failed to write outbox event", e);
    }
}
}

```

File: `src/main/java/com/subsflow/common/context/TenantContext.java`

```

package com.subsflow.common.context;

public class TenantContext {

    private static final ThreadLocal<String> CURRENT_TENANT = new ThreadLocal<>();

    private TenantContext() {
        // Prevent instantiation
    }

    public static void setTenantId(String tenantId) {
        CURRENT_TENANT.set(tenantId);
    }

    public static String getTenantId() {
        return CURRENT_TENANT.get();
    }

    public static void clear() {
        CURRENT_TENANT.remove();
    }
}

```

File: `src/main/java/com/subsflow/common/entity/AbstractTenantEntity.java`

```

package com.subsflow.common.entity;

import jakarta.persistence.Column;
import jakarta.persistence.MappedSuperclass;
import lombok.Getter;
import lombok.Setter;
import org.hibernate.annotations.TenantId;

@MappedSuperclass
@Getter
@Setter
public abstract class AbstractTenantEntity {

    @TenantId
    @Column(name = "tenant_id", nullable = false, updatable = false)
    private String tenantId;
}

```

File: `src/main/java/com/subsflow/common/entity/OutboxEventStatus.java`

```

package com.subsflow.common.entity;

public enum OutboxEventStatus {
    PENDING,
    PROCESSED,
    FAILED
}

```

File: `src/main/java/com/subsflow/common/entity/OutboxEvent.java`

```
package com.subsflow.common.entity;

import jakarta.persistence.*;
import lombok.Getter;
import lombok.Setter;

import java.time.OffsetDateTime;

@Entity
@Table(name = "outbox_events")
@Getter
@Setter
public class OutboxEvent extends AbstractTenantEntity {

    @Id
    private String id;

    @Column(name = "event_type", nullable = false)
    private String eventType;

    @Column(nullable = false, columnDefinition = "TEXT")
    private String payload;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private OutboxEventStatus status = OutboxEventStatus.PENDING;

    @Column(name = "created_at", nullable = false, updatable = false)
    private OffsetDateTime createdAt = OffsetDateTime.now();

    @Column(name = "updated_at", nullable = false)
    private OffsetDateTime updatedAt = OffsetDateTime.now();

    @PreUpdate
    protected void onUpdate() {
        updatedAt = OffsetDateTime.now();
    }
}
```

File: `src/main/java/com/subsflow/common/repository/OutboxEventRepository.java`

```
package com.subsflow.common.repository;

import com.subsflow.common.entity.OutboxEvent;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;

import java.util.List;

public interface OutboxEventRepository extends JpaRepository<OutboxEvent, String> {

    @Query(value = "SELECT * FROM outbox_events WHERE status = 'PENDING' ORDER BY created_at ASC LIMIT 50 FOR UPDATE SKIP LOCKED", nativeQuery = true)
    List<OutboxEvent> findPendingEventsForProcessing();
}
```

File: [src/main/java/com/subsflow/common/outbox/KafkaTopicConfig.java](#)

```
package com.subsflow.common.outbox;

import org.apache.kafka.clients.admin.NewTopic;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.kafka.config.TopicBuilder;

@Configuration
public class KafkaTopicConfig {

    @Bean
    public NewTopic paymentSucceededTopic() {
        return TopicBuilder.name("payment.succeeded").partitions(1).replicas(1).build();
    }

    @Bean
    public NewTopic paymentFailedTopic() {
        return TopicBuilder.name("payment.failed").partitions(1).replicas(1).build();
    }

    @Bean
    public NewTopic subscriptionChangedTopic() {
        return TopicBuilder.name("subscription.changed").partitions(1).replicas(1).build();
    }
}
```


File: src/main/java/com/subsflow/common/outbox/OutboxPublisher.java

```
package com.subsflow.common.outbox;

import com.subsflow.common.context.TenantContext;
import com.subsflow.common.entity.OutboxEvent;
import com.subsflow.common.entity.OutboxEventStatus;
import com.subsflow.common.repository.OutboxEventRepository;
import com.subsflow.tenant.entity.Tenant;
import com.subsflow.tenant.repository.TenantRepository;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.kafka.core.KafkaTemplate;
import org.springframework.scheduling.annotation.EnableScheduling;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Component;
import org.springframework.transaction.annotation.Propagation;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;

@Component
@EnableScheduling
public class OutboxPublisher {

    private static final Logger log = LoggerFactory.getLogger(OutboxPublisher.class);

    private final TenantRepository tenantRepository;
    private final OutboxEventRepository outboxEventRepository;
    private final KafkaTemplate<String, String> kafkaTemplate;

    public OutboxPublisher(TenantRepository tenantRepository,
                          OutboxEventRepository outboxEventRepository,
                          KafkaTemplate<String, String> kafkaTemplate) {
        this.tenantRepository = tenantRepository;
        this.outboxEventRepository = outboxEventRepository;
        this.kafkaTemplate = kafkaTemplate;
    }

    @Scheduled(fixedDelay = 1000)
    public void publishEvents() {
        List<Tenant> tenants = tenantRepository.findAll();
        for (Tenant tenant : tenants) {
            try {
                processTenantEvents(tenant.getId());
            } catch (Exception e) {
                log.error("Error publishing events for tenant: {}", tenant.getId(), e);
            }
        }
    }

    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public void processTenantEvents(String tenantId) {
        try {
            TenantContext.setTenantId(tenantId);
            List<OutboxEvent> pendingEvents = outboxEventRepository.findPendingEventsForProcessing();

            if (pendingEvents.isEmpty()) {
                return;
            }

            log.info("Processing {} outbox events for tenant: {}", pendingEvents.size(), tenantId);

            for (OutboxEvent event : pendingEvents) {
                try {
                    // Resolve topic name from event type
                    String topic = resolveTopic(event.getEventType());

                    try {
                        kafkaTemplate.send(topic, event.getPayload()).get();
                    } catch (Exception publishException) {
                        log.warn("Kafka unavailable for event {}. Leaving it pending for retry", event.getId(),
publishException);
                        throw publishException;
                    }

                    event.setStatus(OutboxEventStatus.PROCESSED);
                    outboxEventRepository.save(event);
                    log.info("Successfully published outbox event: {} to topic: {}", event.getId(), topic);
                }
            }
        }
    }
}
```

```

        } catch (Exception e) {
            log.error("Failed to publish outbox event: {}", event.getId(), e);
            event.setStatus(OutboxEventStatus.FAILED);
            outboxEventRepository.save(event);
        }
    } finally {
        TenantContext.clear();
    }
}

private String resolveTopic(String eventType) {
    // Map event types to Kafka topics
    if ("payment.succeeded".equals(eventType)) {
        return "payment.succeeded";
    } else if ("payment.failed".equals(eventType)) {
        return "payment.failed";
    } else if ("subscription.changed".equals(eventType)) {
        return "subscription.changed";
    }
    return "general.events";
}
}

```

File: `src/main/java/com/subsflow/common/outbox/NotificationStubConsumer.java`

```

package com.subsflow.common.outbox;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.kafka.annotation.KafkaListener;
import org.springframework.stereotype.Component;

import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;

@Component
public class NotificationStubConsumer {

    private static final Logger log = LoggerFactory.getLogger(NotificationStubConsumer.class);
    private final Set<String> seenMessages = ConcurrentHashMap.newKeySet();

    @KafkaListener(topics = "payment.succeeded", groupId = "subsflow-group")
    public void consumePaymentSuccess(String message) {
        if (!seenMessages.add(message)) {
            log.info("Duplicate payment.succeeded event ignored: {}", message);
            return;
        }
        log.info("[STUB CONSUMER] Received payment.succeeded event: {}", message);
    }

    @KafkaListener(topics = "payment.failed", groupId = "subsflow-group")
    public void consumePaymentFailure(String message) {
        if (!seenMessages.add(message)) {
            log.info("Duplicate payment.failed event ignored: {}", message);
            return;
        }
        log.info("[STUB CONSUMER] Received payment.failed event: {}", message);
    }

    @KafkaListener(topics = "subscription.changed", groupId = "subsflow-group")
    public void consumeSubscriptionChange(String message) {
        if (!seenMessages.add(message)) {
            log.info("Duplicate subscription.changed event ignored: {}", message);
            return;
        }
        log.info("[STUB CONSUMER] Received subscription.changed event: {}", message);
    }
}

```

File: src/main/java/com/subsflow/common/filter/TenantAuthFilter.java

```
package com.subsflow.common.filter;

import com.subsflow.common.context.TenantContext;
import com.subsflow.tenant.entity.Tenant;
import com.subsflow.tenant.entity.TenantStatus;
import com.subsflow.tenant.repository.TenantRepository;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.http.HttpStatus;
import org.springframework.http.MediaType;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;
import java.util.Optional;

@Component
public class TenantAuthFilter extends OncePerRequestFilter {

    private final TenantRepository tenantRepository;

    public TenantAuthFilter(TenantRepository tenantRepository) {
        this.tenantRepository = tenantRepository;
    }

    @Override
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response, FilterChain filterChain)
        throws ServletException, IOException {

        String path = request.getRequestURI();

        // Bypass check for public paths
        if (isPublicPath(path, request.getMethod())) {
            filterChain.doFilter(request, response);
            return;
        }

        String apiKey = request.getHeader("X-API-Key");
        if (apiKey == null || apiKey.trim().isEmpty()) {
            writeErrorResponse(response, HttpStatus.UNAUTHORIZED, "Missing X-API-Key header");
            return;
        }

        Optional<Tenant> tenantOpt = tenantRepository.findByApiKey(apiKey);
        if (tenantOpt.isEmpty()) {
            writeErrorResponse(response, HttpStatus.UNAUTHORIZED, "Invalid X-API-Key");
            return;
        }

        Tenant tenant = tenantOpt.get();
        if (tenant.getStatus() == TenantStatus.SUSPENDED) {
            writeErrorResponse(response, HttpStatus.FORBIDDEN, "Tenant account is suspended");
            return;
        }

        try {
            TenantContext.setTenantId(tenant.getId());
            filterChain.doFilter(request, response);
        } finally {
            TenantContext.clear();
        }
    }

    private boolean isPublicPath(String path, String method) {
        // Public tenant onboarding and login routes
        if (("api/v1/tenants".equals(path) || "/api/v1/tenants/login".equals(path)) && ("POST".equalsIgnoreCase(method) ||
"OPTIONS".equalsIgnoreCase(method))) {
            return true;
        }
        // Actuator endpoints are public (health, prometheus, etc.)
        if (path.startsWith("/actuator")) {
            return true;
        }
        if ("OPTIONS".equalsIgnoreCase(method)) {
            return true;
        }
    }
}
```

```

    }
    // For swagger / OpenAPI if needed
    if (path.startsWith("/swagger-ui") || path.startsWith("/v3/api-docs")) {
        return true;
    }
    return false;
}

private void writeErrorResponse(HttpServletResponse response, HttpStatus status, String message) throws IOException {
    response.setStatus(status.value());
    response.setContentType(MediaType.APPLICATION_JSON_VALUE);
    String json = String.format("{\"error\": \"%s\", \"status\": %d}", message, status.value());
    response.getWriter().write(json);
}
}

```

File: `src/main/java/com/subsflow/common/config/CorsConfig.java`

```

package com.subsflow.common.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.CorsRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration
public class CorsConfig {

    @Bean
    public WebMvcConfigurer corsConfigurer() {
        return new WebMvcConfigurer() {
            @Override
            public void addCorsMappings(CorsRegistry registry) {
                registry.addMapping("/")
                    .allowedOrigins("")
                    .allowedMethods("GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS")
                    .allowedHeaders("")
                    .allowCredentials(false);
            }
        };
    }
}

```

File: `src/main/java/com/subsflow/common/config/TenantConnectionProvider.java`

```
package com.subsflow.common.config;

import org.hibernate.cfg.AvailableSettings;
import org.hibernate.engine.jdbc.connections.spi.MultiTenantConnectionProvider;
import org.springframework.boot.autoconfigure.orm.jpa.HibernatePropertiesCustomizer;
import org.springframework.stereotype.Component;

import javax.sql.DataSource;
import java.sql.Connection;
import java.sql.SQLException;
import java.sql.Statement;
import java.util.Map;

@Component
public class TenantConnectionProvider implements MultiTenantConnectionProvider, HibernatePropertiesCustomizer {

    private final DataSource dataSource;

    public TenantConnectionProvider(DataSource dataSource) {
        this.dataSource = dataSource;
    }

    @Override
    public Connection getAnyConnection() throws SQLException {
        return dataSource.getConnection();
    }

    @Override
    public void releaseAnyConnection(Connection connection) throws SQLException {
        connection.close();
    }

    @Override
    public Connection getConnection(Object tenantIdentifier) throws SQLException {
        Connection connection = dataSource.getConnection();
        String tenantId = tenantIdentifier != null ? tenantIdentifier.toString() : null;
        if (tenantId != null && !TenantIdentifierResolver.BOOTSTRAP_TENANT.equals(tenantId)) {
            if (!tenantId.matches("[a-zA-Z0-9_-]+$")) {
                throw new IllegalArgumentException("Invalid tenant identifier format");
            }
            try (Statement stmt = connection.createStatement()) {
                stmt.execute("SET app.current_tenant_id = '" + tenantId + "'");
            } catch (SQLException e) {
                connection.close();
                throw e;
            }
        }
        return connection;
    }

    @Override
    public void releaseConnection(Object tenantIdentifier, Connection connection) throws SQLException {
        String tenantId = tenantIdentifier != null ? tenantIdentifier.toString() : null;
        if (tenantId != null && !TenantIdentifierResolver.BOOTSTRAP_TENANT.equals(tenantId)) {
            try (Statement stmt = connection.createStatement()) {
                stmt.execute("RESET app.current_tenant_id");
            } catch (SQLException e) {
                connection.close();
                throw e;
            }
        }
        connection.close();
    }

    @Override
    public boolean supportsAggressiveRelease() {
        return false;
    }

    @Override
    public boolean isUnwrappableAs(Class<?> unwrapType) {
        return false;
    }

    @Override
    public <T> T unwrap(Class<T> unwrapType) {
        return null;
    }
}
```

```
    }

    @Override
    public void customize(Map<String, Object> hibernateProperties) {
        hibernateProperties.put(AvailableSettings.MULTI_TENANT_CONNECTION_PROVIDER, this);
    }
}
```

File: `src/main/java/com/subsflow/common/config/TenantIdentifierResolver.java`

```
package com.subsflow.common.config;

import com.subsflow.common.context.TenantContext;
import org.hibernate.cfg.AvailableSettings;
import org.hibernate.context.spi.CurrentTenantIdentifierResolver;
import org.springframework.boot.autoconfigure.orm.jpa.HibernatePropertiesCustomizer;
import org.springframework.stereotype.Component;

import java.util.Map;

@Component
public class TenantIdentifierResolver implements CurrentTenantIdentifierResolver<String>, HibernatePropertiesCustomizer {

    public static final String BOOTSTRAP_TENANT = "bootstrap";

    @Override
    public String resolveCurrentTenantIdentifier() {
        String tenantId = TenantContext.getTenantId();
        return tenantId != null ? tenantId : BOOTSTRAP_TENANT;
    }

    @Override
    public boolean validateExistingCurrentSessions() {
        return true;
    }

    @Override
    public void customize(Map<String, Object> hibernateProperties) {
        hibernateProperties.put(AvailableSettings.MULTI_TENANT_IDENTIFIER_RESOLVER, this);
    }
}
```

File: `src/main/java/com/subsflow/common/metrics/MetricsService.java`

```
package com.subsflow.common.metrics;

import io.micrometer.core.instrument.Counter;
import io.micrometer.core.instrument.MeterRegistry;
import org.springframework.stereotype.Component;

import java.util.concurrent.atomic.AtomicInteger;
import java.util.concurrent.atomic.AtomicReference;

@Component
public class MetricsService {

    private final Counter paymentSuccessCounter;
    private final Counter paymentFailedCounter;
    private final Counter dunningRetryCounter;
    private final AtomicReference<Double> activeMrr = new AtomicReference<> (0.0);
    private final AtomicInteger activeSubscriptions = new AtomicInteger(0);

    public MetricsService(MeterRegistry registry) {
        this.paymentSuccessCounter = Counter.builder("subsflow.payment.success")
            .description("Total number of successful payment transactions")
            .register(registry);

        this.paymentFailedCounter = Counter.builder("subsflow.payment.failed")
            .description("Total number of failed payment transactions")
            .register(registry);

        this.dunningRetryCounter = Counter.builder("subsflow.dunning.retries")
            .description("Total number of payment dunning retries executed")
            .register(registry);

        registry.gauge("subsflow.mrr", activeMrr, AtomicReference::get);
        registry.gauge("subsflow.active.subscriptions", activeSubscriptions, AtomicInteger::get);
    }

    public void incrementPaymentSuccess() {
        paymentSuccessCounter.increment();
    }

    public void incrementPaymentFailed() {
        paymentFailedCounter.increment();
    }

    public void incrementDunningRetry() {
        dunningRetryCounter.increment();
    }

    public void updateMrr(double mrrValue) {
        activeMrr.set(mrrValue);
    }

    public void setActiveSubscriptions(int count) {
        activeSubscriptions.set(count);
    }
}
```

File: `src/main/java/com/subsflow/common/exception/IdempotencyConflictException.java`

```
package com.subsflow.common.exception;

public class IdempotencyConflictException extends RuntimeException {
    public IdempotencyConflictException(String message) {
        super(message);
    }
}
```

File: [src/main/java/com/subsflow/common/exception/IdempotencyKeyUsedException.java](#)

```
package com.subsflow.common.exception;

public class IdempotencyKeyUsedException extends RuntimeException {

    private final String responsePayload;

    public IdempotencyKeyUsedException(String message, String responsePayload) {
        super(message);
        this.responsePayload = responsePayload;
    }

    public String getResponsePayload() {
        return responsePayload;
    }
}
```


File: github.com/subsflow/tenant/TenantIsolationTest.java

```
package com.subsflow.tenant;

import com.subsflow.common.context.TenantContext;
import com.subsflow.subscription.entity.BillingPeriod;
import com.subsflow.subscription.entity.BillingPlan;
import com.subsflow.subscription.entity.BillingType;
import com.subsflow.subscription.repository.BillingPlanRepository;
import com.subsflow.tenant.entity.Tenant;
import com.subsflow.tenant.entity.TenantStatus;
import com.subsflow.tenant.repository.TenantRepository;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.test.context.DynamicPropertyRegistry;
import org.springframework.test.context.DynamicPropertySource;
import org.testcontainers.containers.PostgreSQLContainer;
import org.testcontainers.junit.jupiter.Container;
import org.testcontainers.junit.jupiter.Testcontainers;

import java.math.BigDecimal;
import java.util.List;

import static org.assertj.core.api.Assertions.assertThat;

@SpringBootTest(properties = {
    "spring.autoconfigure.exclude=org.springframework.boot.autoconfigure.data.redis.RedisAutoConfiguration,org.springframework.boot.autoconfigure.kafka.KafkaAutoConfiguration"
})
@Testcontainers
public class TenantIsolationTest {

    @Container
    static PostgreSQLContainer<> postgres = new PostgreSQLContainer<>("postgres:15-alpine");

    @DynamicPropertySource
    static void configureProperties(DynamicPropertyRegistry registry) {
        registry.add("spring.datasource.url", postgres::getJdbcUrl);
        registry.add("spring.datasource.username", postgres::getUsername);
        registry.add("spring.datasource.password", postgres::getPassword);
    }

    @Autowired
    private TenantRepository tenantRepository;

    @Autowired
    private BillingPlanRepository billingPlanRepository;

    private Tenant tenantA;
    private Tenant tenantB;

    @BeforeEach
    void setUp() {
        // Clear database tables (order is important due to constraints)
        billingPlanRepository.deleteAll();
        tenantRepository.deleteAll();

        // Create Tenant A
        tenantA = new Tenant();
        tenantA.setId("tenant-a");
        tenantA.setName("Tenant A");
        tenantA.setStatus(TenantStatus.ACTIVE);
        tenantA.setApiKey("key-a");
        tenantRepository.save(tenantA);

        // Create Tenant B
        tenantB = new Tenant();
        tenantB.setId("tenant-b");
        tenantB.setName("Tenant B");
        tenantB.setStatus(TenantStatus.ACTIVE);
        tenantB.setApiKey("key-b");
        tenantRepository.save(tenantB);
    }

    @Test
    void testTenantIsolationWithJPA() {
        // 1. Create a billing plan under Tenant A
    }
}
```

```
TenantContext.setTenantId(tenantA.getId());
BillingPlan planA = new BillingPlan();
planA.setId("plan-a");
planA.setName("Plan A (Tenant A)");
planA.setBillingType(BillingType.FIXED);
planA.setPrice(new BigDecimal("99.99"));
planA.setBillingPeriod(BillingPeriod.MONTHLY);
billingPlanRepository.save(planA);
TenantContext.clear();

// 2. Create a billing plan under Tenant B
TenantContext.setTenantId(tenantB.getId());
BillingPlan planB = new BillingPlan();
planB.setId("plan-b");
planB.setName("Plan B (Tenant B)");
planB.setBillingType(BillingType.USAGE_BASED);
planB.setPrice(new BigDecimal("0.05"));
planB.setBillingPeriod(BillingPeriod.MONTHLY);
billingPlanRepository.save(planB);
TenantContext.clear();

// 3. Under Tenant A Context: query plan. Assert only Plan A is visible.
TenantContext.setTenantId(tenantA.getId());
List<BillingPlan> plansA = billingPlanRepository.findAll();
assertThat(plansA).hasSize(1);
assertThat(plansA.get(0).getId()).isEqualTo("plan-a");
TenantContext.clear();

// 4. Under Tenant B Context: query plan. Assert only Plan B is visible.
TenantContext.setTenantId(tenantB.getId());
List<BillingPlan> plansB = billingPlanRepository.findAll();
assertThat(plansB).hasSize(1);
assertThat(plansB.get(0).getId()).isEqualTo("plan-b");
TenantContext.clear();
}
```

File: <https://github.com/subsflow/subsflow/blob/master/subsflow-subscription/src/test/java/com/subsflow/subscription/SubscriptionOptimisticLockingTest.java>

```
package com.subsflow.subscription;

import com.subsflow.common.context.TenantContext;
import com.subsflow.subscription.entity.BillingPeriod;
import com.subsflow.subscription.entity.BillingPlan;
import com.subsflow.subscription.entity.BillingType;
import com.subsflow.subscription.entity.Subscription;
import com.subsflow.subscription.entity.SubscriptionStatus;
import com.subsflow.subscription.repository.BillingPlanRepository;
import com.subsflow.subscription.repository.SubscriptionRepository;
import com.subsflow.tenant.entity.Tenant;
import com.subsflow.tenant.entity.TenantStatus;
import com.subsflow.tenant.repository.TenantRepository;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.orm.ObjectOptimisticLockingFailureException;
import org.springframework.test.context.DynamicPropertyRegistry;
import org.springframework.test.context.DynamicPropertySource;
import org.testcontainers.containers.PostgreSQLContainer;
import org.testcontainers.junit.jupiter.Container;
import org.testcontainers.junit.jupiter.Testcontainers;

import java.math.BigDecimal;
import java.time.OffsetDateTime;

import static org.assertj.core.api.Assertions.assertThatThrownBy;

@SpringBootTest(properties = {
    "spring.autoconfigure.exclude=org.springframework.boot.autoconfigure.data.redis.RedisAutoConfiguration,org.springframework.boot.autoconfigure.kafka.KafkaAutoConfiguration"
})
@Testcontainers
class SubscriptionOptimisticLockingTest {

    @Container
    static PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>("postgres:15-alpine");

    @DynamicPropertySource
    static void configureProperties(DynamicPropertyRegistry registry) {
        registry.add("spring.datasource.url", postgres::getJdbcUrl);
        registry.add("spring.datasource.username", postgres::getUsername);
        registry.add("spring.datasource.password", postgres::getPassword);
    }

    @Autowired
    private TenantRepository tenantRepository;

    @Autowired
    private BillingPlanRepository billingPlanRepository;

    @Autowired
    private SubscriptionRepository subscriptionRepository;

    private Tenant tenant;

    @BeforeEach
    void setUp() {
        subscriptionRepository.deleteAll();
        billingPlanRepository.deleteAll();
        tenantRepository.deleteAll();

        tenant = new Tenant();
        tenant.setId("tenant-opt");
        tenant.setName("Opt Tenant");
        tenant.setStatus(TenantStatus.ACTIVE);
        tenant.setApiKey("key-opt");
        tenantRepository.save(tenant);
    }

    @Test
    void concurrentUpdatesToSameSubscriptionShouldFailOptimistically() {
        TenantContext.setTenantId(tenant.getId());
        try {
            BillingPlan planA = new BillingPlan();
            planA.setId("plan-a");
        }
    }
}
```

```

        planA.setName("Plan A");
        planA.setBillingType(BillingType.FIXED);
        planA.setPrice(new BigDecimal("10.00"));
        planA.setBillingPeriod(BillingPeriod.MONTHLY);
        billingPlanRepository.save(planA);

        BillingPlan planB = new BillingPlan();
        planB.setId("plan-b");
        planB.setName("Plan B");
        planB.setBillingType(BillingType.FIXED);
        planB.setPrice(new BigDecimal("20.00"));
        planB.setBillingPeriod(BillingPeriod.MONTHLY);
        billingPlanRepository.save(planB);

        Subscription subscription = new Subscription();
        subscription.setId("sub-1");
        subscription.setPlan(planA);
        subscription.setStatus(SubscriptionStatus.ACTIVE);
        subscription.setCurrentPeriodStart(OffsetDateTime.now().minusDays(10));
        subscription.setCurrentPeriodEnd(OffsetDateTime.now().plusDays(20));
        subscriptionRepository.save(subscription);

        Subscription stale = subscriptionRepository.findById("sub-1").orElseThrow();
        Subscription fresh = subscriptionRepository.findById("sub-1").orElseThrow();
        stale.setPlan(planA);
        stale.setStatus(SubscriptionStatus.PAST_DUE);
        subscriptionRepository.saveAndFlush(stale);

        fresh.setPlan(planB);
        fresh.setStatus(SubscriptionStatus.ACTIVE);

        assertThatThrownBy(() -> subscriptionRepository.saveAndFlush(fresh))
            .isInstanceOf(ObjectOptimisticLockingFailureException.class);
    } finally {
        TenantContext.clear();
    }
}

```

File: [src/test/java/com/subsflow/subscription/SubscriptionStatusTransitionTest.java](#)

```

package com.subsflow.subscription;

import com.subsflow.subscription.entity.SubscriptionStatus;
import com.subsflow.subscription.service.SubscriptionStatusTransition;
import org.junit.jupiter.api.Test;

import static org.assertj.core.api.Assertions.assertThatCode;
import static org.assertj.core.api.Assertions.assertThatThrownBy;

class SubscriptionStatusTransitionTest {

    @Test
    void cancelToActiveTransitionIsRejected() {
        assertThatThrownBy(() -> SubscriptionStatusTransition.requireTransition(
            SubscriptionStatus.CANCELLED,
            SubscriptionStatus.ACTIVE,
            "reactivation"
        )).isInstanceOf(IllegalStateException.class)
            .hasMessageContaining("not allowed");
    }

    @Test
    void pastDueToActiveTransitionIsAllowed() {
        assertThatCode(() -> SubscriptionStatusTransition.requireTransition(
            SubscriptionStatus.PAST_DUE,
            SubscriptionStatus.ACTIVE,
            "payment recovered"
        )).doesNotThrowAnyException();
    }
}

```

File: [src/test/java/com/subsflow/payment/IdempotencyServiceTest.java](https://github.com/subsflow/subsflow-architecture-manifest/blob/main/src/test/java/com/subsflow/payment/IdempotencyServiceTest.java)

```
package com.subsflow.payment;

import com.subsflow.common.context.TenantContext;
import com.subsflow.common.exception.IdempotencyConflictException;
import com.subsflow.common.exception.IdempotencyKeyUsedException;
import com.subsflow.payment.repository.IdempotencyKeyRepository;
import com.subsflow.payment.service.IdempotencyService;
import com.subsflow.tenant.entity.Tenant;
import com.subsflow.tenant.entity.TenantStatus;
import com.subsflow.tenant.repository.TenantRepository;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.test.context.DynamicPropertyRegistry;
import org.springframework.test.context.DynamicPropertySource;
import org.testcontainers.containers.PostgreSQLContainer;
import org.testcontainers.junit.jupiter.Container;
import org.testcontainers.junit.jupiter.Testcontainers;

import static org.assertj.core.api.Assertions.assertThatThrownBy;
import static org.assertj.core.api.Assertions.assertThat;

@SpringBootTest(properties = {
    "spring.autoconfigure.exclude=org.springframework.boot.autoconfigure.data.redis.RedisAutoConfiguration,org.springframework.boot.autoconfigure.kafka.KafkaAutoConfiguration"
})
@Testcontainers
public class IdempotencyServiceTest {

    @Container
    static PostgreSQLContainer<> postgres = new PostgreSQLContainer<>("postgres:15-alpine");

    @DynamicPropertySource
    static void configureProperties(DynamicPropertyRegistry registry) {
        registry.add("spring.datasource.url", postgres::getJdbcUrl);
        registry.add("spring.datasource.username", postgres::getUsername);
        registry.add("spring.datasource.password", postgres::getPassword);
    }

    @Autowired
    private TenantRepository tenantRepository;

    @Autowired
    private IdempotencyKeyRepository idempotencyKeyRepository;

    @Autowired
    private IdempotencyService idempotencyService;

    private Tenant tenantA;
    private Tenant tenantB;

    @BeforeEach
    void setUp() {
        idempotencyKeyRepository.deleteAll();
        tenantRepository.deleteAll();

        // Create Tenant A
        tenantA = new Tenant();
        tenantA.setId("tenant-a");
        tenantA.setName("Tenant A");
        tenantA.setStatus(TenantStatus.ACTIVE);
        tenantA.setApiKey("key-a");
        tenantRepository.save(tenantA);

        // Create Tenant B
        tenantB = new Tenant();
        tenantB.setId("tenant-b");
        tenantB.setName("Tenant B");
        tenantB.setStatus(TenantStatus.ACTIVE);
        tenantB.setApiKey("key-b");
        tenantRepository.save(tenantB);
    }

    @Test
    void testIdempotencyLifecycle() {
        // 1. Tenant A starts operation
    }
}
```

```
TenantContext.setTenantId(tenantA.getId());
idempotencyService.startOperation("req-key-1", "hash-value-1");

// 2. Parallel/duplicate request with same key & hash should throw conflict (in progress)
assertThatThrownBy(() -> idempotencyService.startOperation("req-key-1", "hash-value-1"))
    .isInstanceOf(IdempotencyConflictException.class)
    .hasMessageContaining("Operation is already in progress");

// 3. Complete the operation
idempotencyService.completeOperation("req-key-1", "{\"status\":\"success\",\"amount\":99.99}");

// 4. Repeated request should throw IdempotencyKeyUsedException containing the cached payload
assertThatThrownBy(() -> idempotencyService.startOperation("req-key-1", "hash-value-1"))
    .isInstanceOf(IdempotencyKeyUsedException.class)
    .satisfies(ex -> {
        IdempotencyKeyUsedException usedEx = (IdempotencyKeyUsedException) ex;
        assertThat(usedEx.getResponsePayload()).isEqualTo("{\"status\":\"success\",\"amount\":99.99}");
    });

// 5. Querying with a different request hash (key reuse misuse) should throw conflict
assertThatThrownBy(() -> idempotencyService.startOperation("req-key-1", "hash-different"))
    .isInstanceOf(IdempotencyConflictException.class)
    .hasMessageContaining("used with a different request payload");

TenantContext.clear();

// 6. Tenant B uses the same key -> should succeed (tenant isolated!)
TenantContext.setTenantId(tenantB.getId());
idempotencyService.startOperation("req-key-1", "hash-value-1"); // Should not clash with tenant A
TenantContext.clear();
    }
}
```

File: [src/test/java/com/subsflow/document/PaymentGatewayConcurrencyTest.java](https://github.com/subsflow/subsflow/blob/master/payment-gateway-concurrency-test.java)

```
package com.subsflow.payment;

import com.subsflow.billing.entity.Invoice;
import com.subsflow.billing.entity.InvoiceStatus;
import com.subsflow.payment.entity.PaymentStatus;
import com.subsflow.payment.entity.PaymentTransaction;
import com.subsflow.payment.service.PaymentService;
import com.subsflow.subscription.entity.Subscription;
import com.subsflow.subscription.entity.SubscriptionStatus;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.boot.test.mock.mockito.MockBean;
import org.springframework.test.context.DynamicPropertyRegistry;
import org.springframework.test.context.DynamicPropertySource;
import org.testcontainers.containers.PostgreSQLContainer;
import org.testcontainers.junit.jupiter.Container;
import org.testcontainers.junit.jupiter.Testcontainers;

import java.math.BigDecimal;
import java.time.OffsetDateTime;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;

import static org.assertj.core.api.Assertions.assertThat;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.when;

@SpringBootTest(properties = {
    "spring.autoconfigure.exclude=org.springframework.boot.autoconfigure.data.redis.RedisAutoConfiguration,org.springframework.boot.autoconfigure.kafka.KafkaAutoConfiguration"
})
@Testcontainers
class PaymentGatewayConcurrencyTest {

    @Container
    static PostgreSQLContainer<> postgres = new PostgreSQLContainer<>("postgres:15-alpine");

    @DynamicPropertySource
    static void configureProperties(DynamicPropertyRegistry registry) {
        registry.add("spring.datasource.url", postgres::getJdbcUrl);
        registry.add("spring.datasource.username", postgres::getUsername);
        registry.add("spring.datasource.password", postgres::getPassword);
    }

    @Autowired
    private PaymentService paymentService;

    @MockBean
    private com.subsflow.payment.gateway.PaymentGatewayClient gatewayClient;

    @Test
    void duplicateChargeCallsShouldBePreventedByIdempotencyKey() throws Exception {
        // This test exercises the service path for concurrent duplicate payment calls; the actual guard is enforced in the controller/service layer.
        Subscription subscription = new Subscription();
        subscription.setId("sub-dup");
        subscription.setStatus(SubscriptionStatus.ACTIVE);
        subscription.setCurrentPeriodStart(OffsetDateTime.now().minusDays(10));
        subscription.setCurrentPeriodEnd(OffsetDateTime.now().plusDays(20));

        Invoice invoice = new Invoice();
        invoice.setId("inv-dup");
        invoice.setAmount(new BigDecimal("25.00"));
        invoice.setStatus(InvoiceStatus.DRAFT);
        invoice.setDueDate(OffsetDateTime.now().plusDays(7));

        when(gatewayClient.charge(any())).thenReturn(new com.subsflow.payment.gateway.ChargeResponse(true, "gw-1", null));

        ExecutorService pool = Executors.newFixedThreadPool(2);
        CountDownLatch start = new CountDownLatch(1);
        Future<PaymentTransaction> first = pool.submit(() -> {
            start.await();
            return paymentService.processPayment(subscription, invoice, "pm-1", "dup-key");
        });
    }
}
```

```
Future<PaymentTransaction> second = pool.submit(() -> {
    start.await();
    return paymentService.processPayment(subscription, invoice, "pm-1", "dup-key");
});
start.countDown();
PaymentTransaction tx1 = first.get();
PaymentTransaction tx2 = second.get();
pool.shutdown();

assertThat(tx1.getStatus()).isEqualTo(PaymentStatus.SUCCESS);
assertThat(tx2.getStatus()).isEqualTo(PaymentStatus.SUCCESS);
}
```